



UNIVERSITÀ DI PARMA

Image Filtering

Image processing reduces image information.
A processed image can often seem more eye
appealing, but is always poorer than original
one

- What is an Image?
- Image Filtering
- Local Operations
 - Image Convolution
 - Linear filters
 - Smoothing
 - Gradient
 - Padding
 - Stride
 - Non linear filters
 - Bilateral filtering
 - Integral Image

What are images?



What are images?



$$P = f(x, y)$$

$$f : R^2 \Rightarrow R$$

$$P = f(x, y)$$

$$f : R^2 \Rightarrow R$$

1. Signal is **sampled** to obtain a function (r, c)
2. Result is also **quantized** given we need a discrete function
3. One or more **channels** for each pixel

$R(r, c), G(r, c), B(r, c)$

$\text{Luminance}(r, c)$

Image filtering

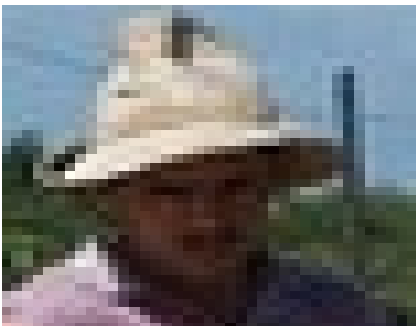
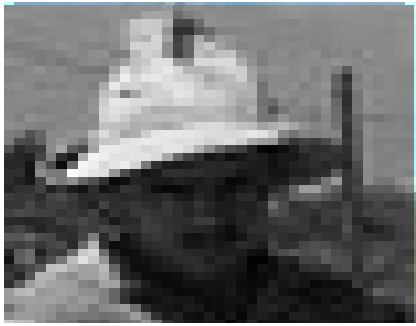
$$F(\text{img}) = \text{img}$$
A color photograph of a boat with a white cabin and a dark hull, sailing on a body of water. The image is slightly blurred.A grayscale version of the same boat image, where the colors have been converted to shades of gray, resulting in a high-contrast, somewhat noisy appearance.

Image filtering

$$F(\text{img}) = \text{img}$$

Image filtering



$$F(\text{Image}) =$$

0.1
0
0.8
0.9
0.9
0.9
0.2
0.4
0.3
0.6
0
0
0.1
0.5
0.9
0.9
0.2
0.4
0.3
0.6
0
0
0.1
0.9
0.9
0.2
0.4
0.3
0.6

Image filtering

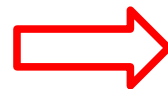
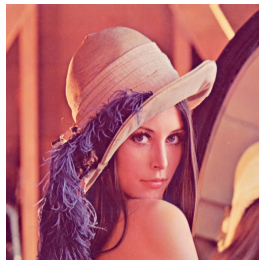
$$F(\text{Image 1}, \text{Image 2}) = \text{Image 3}$$

The equation illustrates an image filtering operation. The first image is a blurry, low-resolution photo of a person wearing a wide-brimmed straw hat. The second image is a sharper, higher-resolution photo of a brown horse's head. The result of the operation, indicated by the equals sign, is a third image that is a blurred version of the horse's head, demonstrating the effect of a low-pass filter.

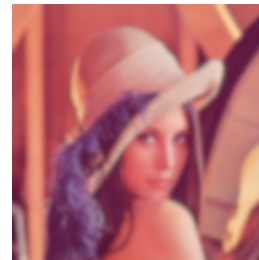
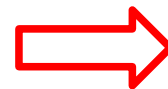
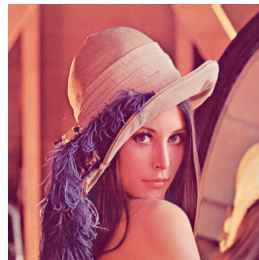
- Filters
 - Mathematical functions that operate on the pixels
 - Often Image $\rightarrow$ Image
- Different classes
 - **Point operations:** result depends only on each pixel value
 - **Local operations:** result depends on each pixel + neighborhood values
 - **Global operations:** result depends on the values of all image pixels

Image Filtering Examples

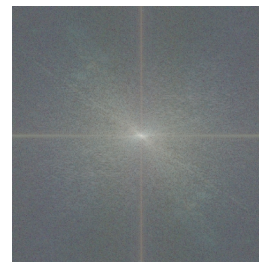
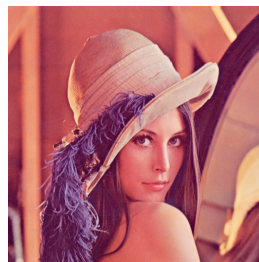
- Point operations:
 - Color to gray levels



- Local operations:
 - Running average (blur)



- Global operations:
 - Fourier transform





UNIVERSITÀ DI PARMA

Image Filtering

Local & Linear operations

Convolution

Local Operations

$$F(\text{Image with neighborhood dots}) = \text{Blurred Image}$$

- Result depends only on
 - Pixel value
 - Pixel Neighborhood

- We already said that an image can be seen as a matrix
 - Each matrix element corresponds to a image point better known as **pixel**
 - Each pixel feature one or more values that allow to define its content
 - Color, luminance, transparency...
 - Not necessarily integer numbers...
- For following slides, we are going to deal with grey level only images
 - Each pixel must encode luminance only
 - Only 1 integer non negative number for each pixel
 - The higher the number, the brighter the pixel
 - $0 \rightarrow$ black

- Image convolution is the main operation used for local image filtering
 - From $f[.,.] \rightarrow h[.,.]$
- $g[k,l]$ is the **Kernel** or Convolution Matrix and represents the pixel neighborhood weight
 - Kernel size

- **Linear operation**

- Symbol: *
 - $h=g*f$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$g[.,.]$

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10							

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$g[.,.]$

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20						

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$g[.,.]$

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30					

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$g[.,.]$

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30				

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$g[.,.]$

$f[.,.]$
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$
$$\mathbf{g}[\cdot, \cdot]$$

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30				
							?		

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$g[.,.]$

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

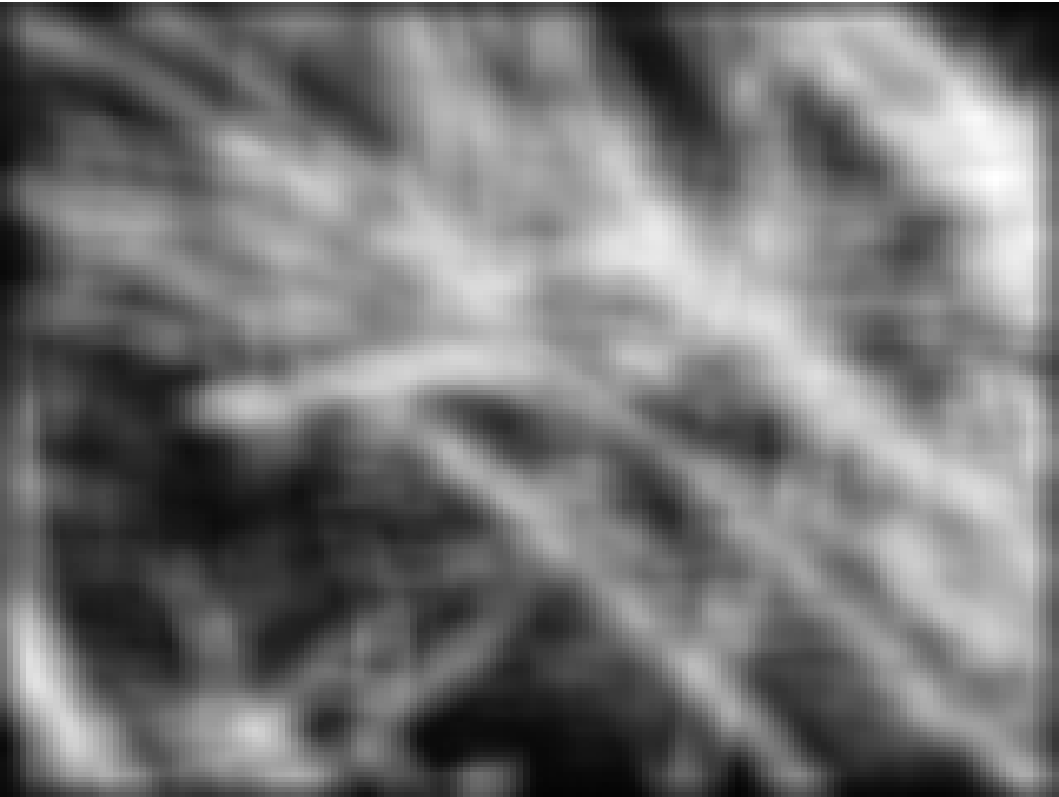
$g[.,.]$

- What is the result of such filtering?
 - Neighborhood average
 - Low pass filter
- Basically sharp transitions are reduced
 - blurring
- **Mean/Box filter smoothing**

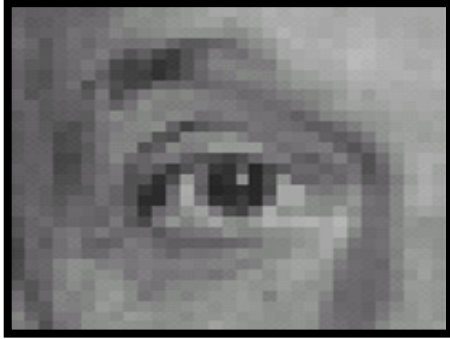
$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

Box Filter Smoothing



Other examples

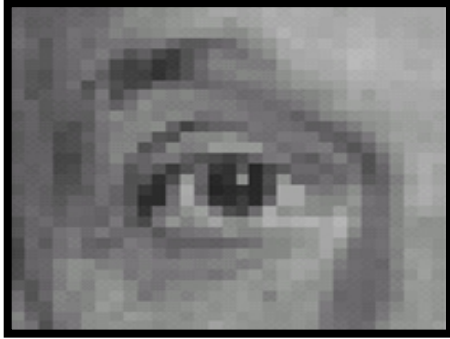


Original

0	0	0
0	1	0
0	0	0

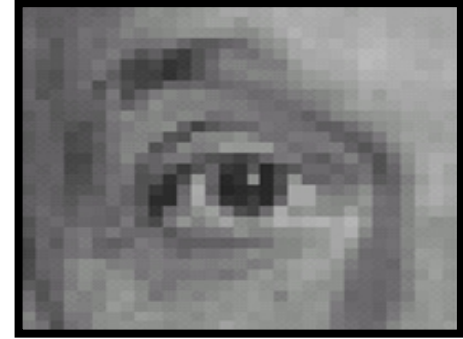
?

Other examples



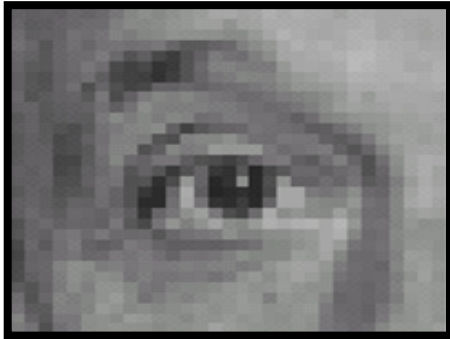
Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Other examples

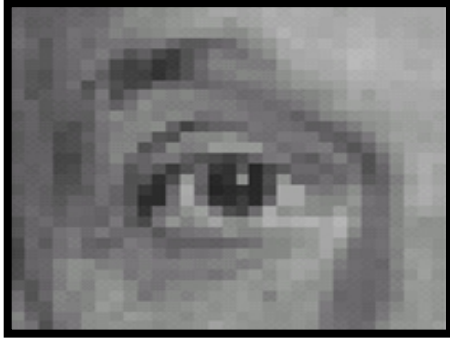


Original

0	0	0
0	0	1
0	0	0

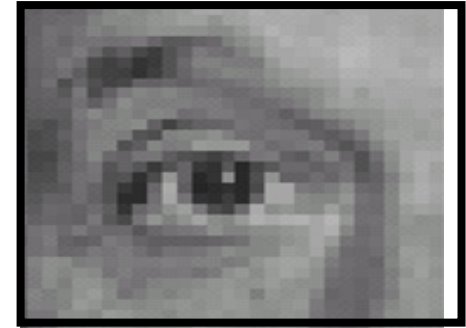
?

Other examples



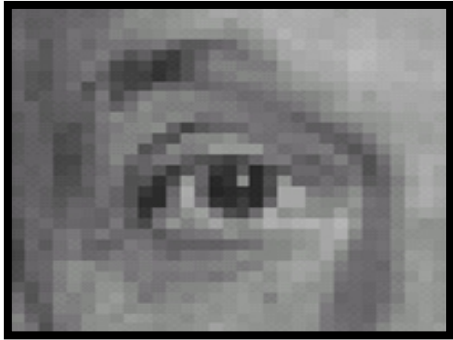
Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Other examples



Original

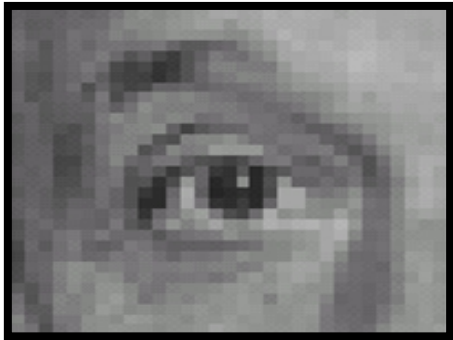
0	0	0
0	2	0
0	0	0

-

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?



Original

0	0	0
0	2	0
0	0	0

−

$\frac{1}{9}$

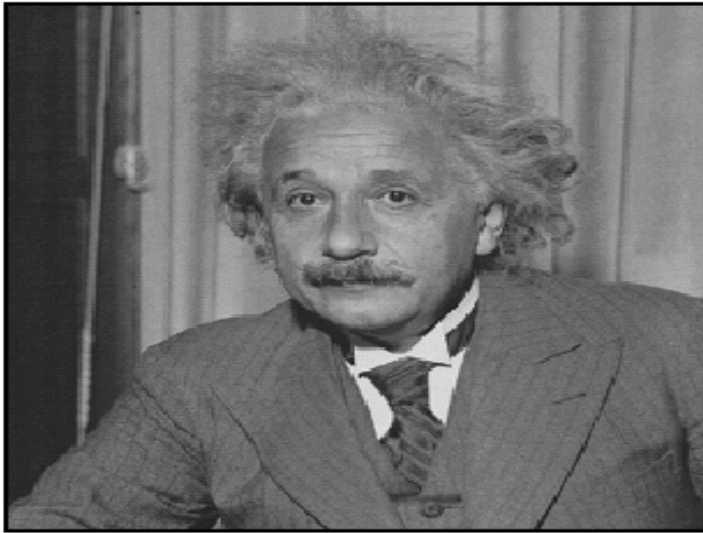
1	1	1
1	1	1
1	1	1



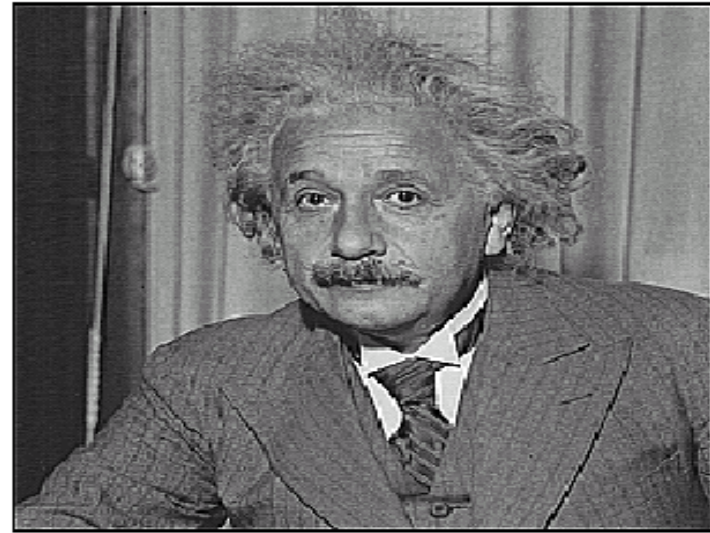
Sharpening filter

- Accentuates differences with local average
- High pass filtering

Other examples

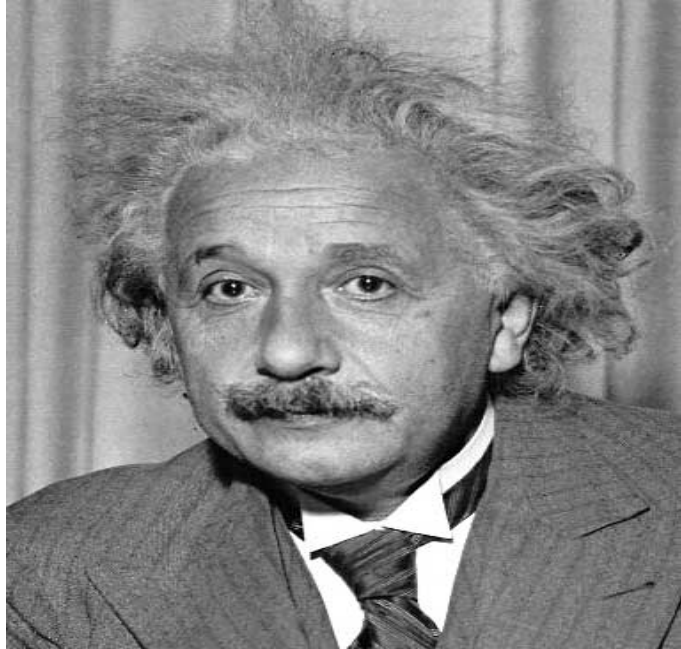


before



after

Slide credit: David Lowe (UBC)

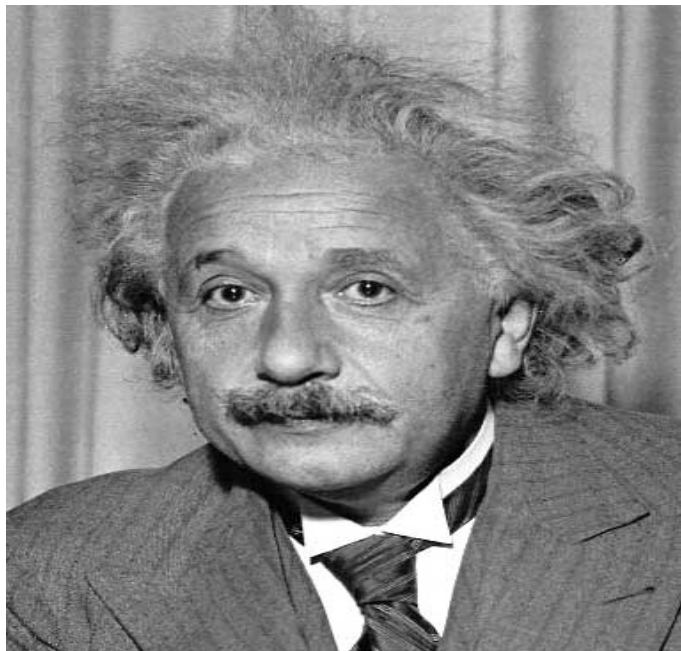


1	0	-1
2	0	-2
1	0	-1

Vertical
Sobel



**Vertical Edge
(absolute value)**



1	2	1
0	0	0
-1	-2	-1

Horizontal
Sobel



**Horizontal Edge
(absolute value)**

Horizontal Gradient

0	0	0
-1	0	1
0	0	0

or

-1	0	1
----	---	---

Vertical Gradient

0	1	0
0	0	0
0	-1	0

or

-1
0
1

- We can have negative values
- Generally pixel channels are unsigned
- How to handle them?
 - Clamping: put negative values as 0
 - Abs: consider absolute values only
 - Shift: add a value (may reduce interval)
 - Use them: adapt internal representation to handle them

- **Some** kernel filters are **separable**
- $n \times n$ kernel in a single convolution
 - $O(M \times M \times n \times n)$
- $1 \times n$ and $n \times 1$ convolutions
 - $O(M \times M \times (n + n))$
- Mean filtering example:

$$\frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * \frac{1}{3} [1 \quad 1 \quad 1] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



UNIVERSITÀ DI PARMA

Image Filtering

main local operations

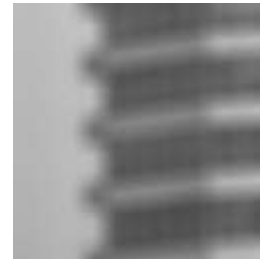
- Image can be affected by noise
 - Sensor
 - Transmission
 - Lossy compressions
 - Filtering can enhance noise (derivative operations)
- Different kind of noises
 - White noise
 - Salt&Pepper
 - ...
- Smoothing is often used to “reduce” it

SMOOTHING: Mean Filtering

- Averaging neighborhood can smooth images
- The bigger the kernel size, the augmented the smooth effect
 - Balancing act
 - Examples with 3x3, 5x5, 9x9, and 11x11 kernels

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



- Pros
 - Easy implementation
- Cons:
 - Weight of each pixel is the same
 - Not good for specific noise effects
 - Introduce high frequency artifacts

SMOOTHING: Salt & Pepper noise



Noisy Image



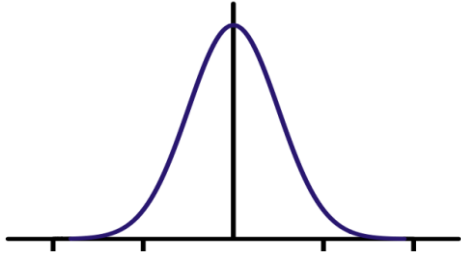
“Ideal” Filtering



Mean Filtering result 👎

- Mean filtering kernel use the same weight for each pixel
 - But the distance is different
- Gaussian filtering gives more weight to the central pixel and:
 - The farther away the neighbors, the smaller their contribution
 - A gaussian is used

SMOOTHING: Gaussian filtering

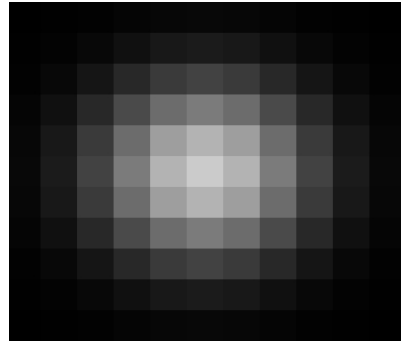


$$g_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



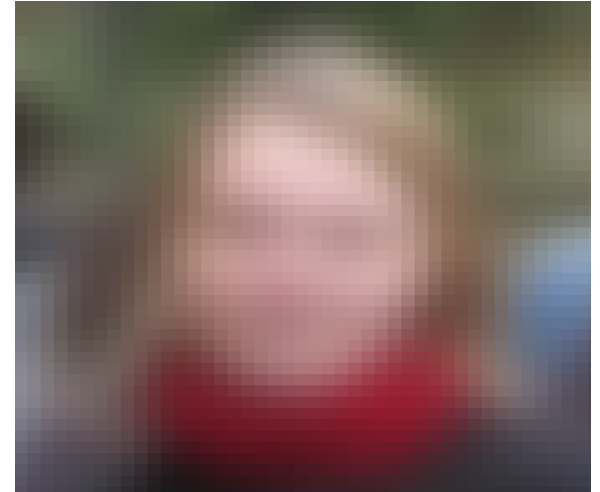
Input image f

*



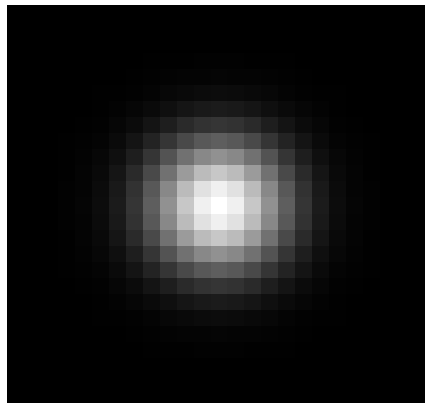
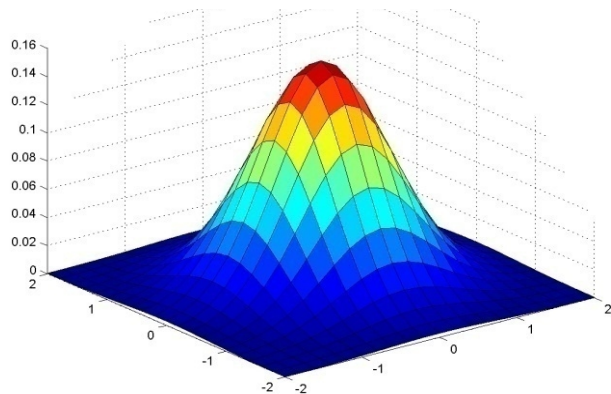
Filter g

=



Output image h

Spatially-weighted average



0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$
(normalized values)

$$g_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

SMOOTHING: Gaussian vs Mean

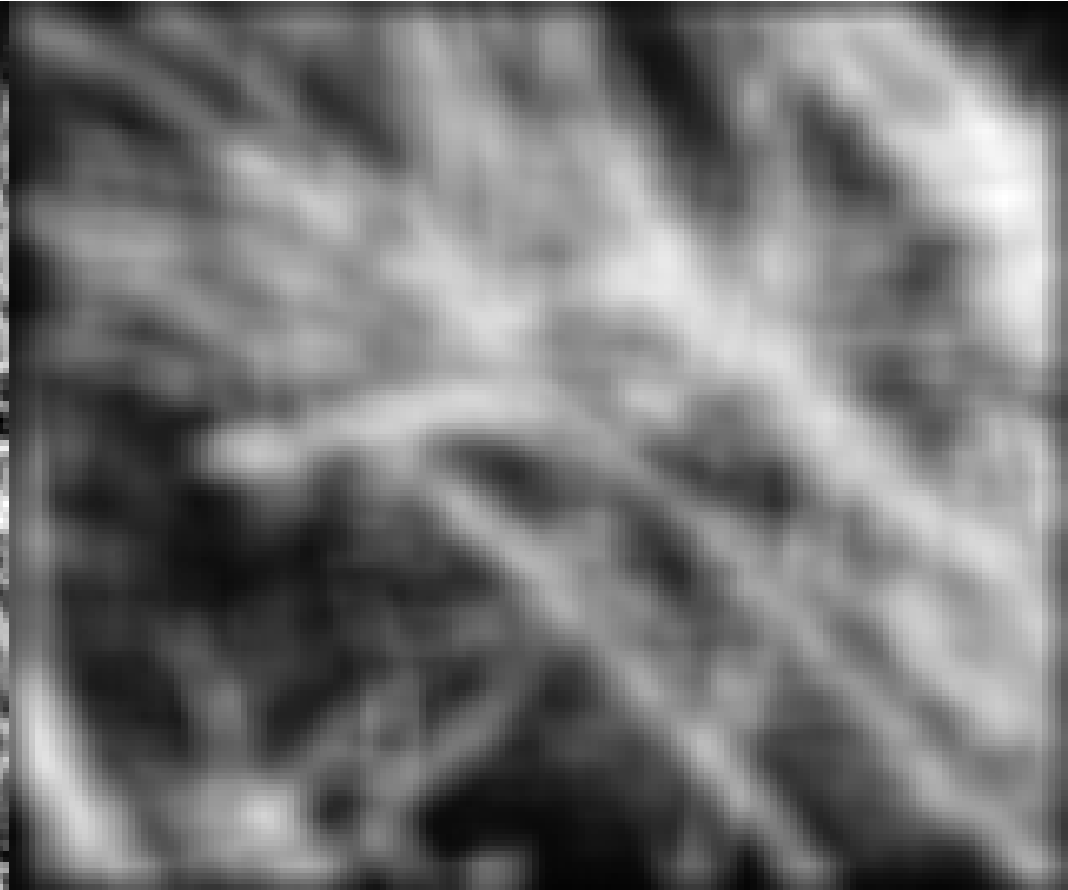


SMOOTHING: Gaussian vs Mean

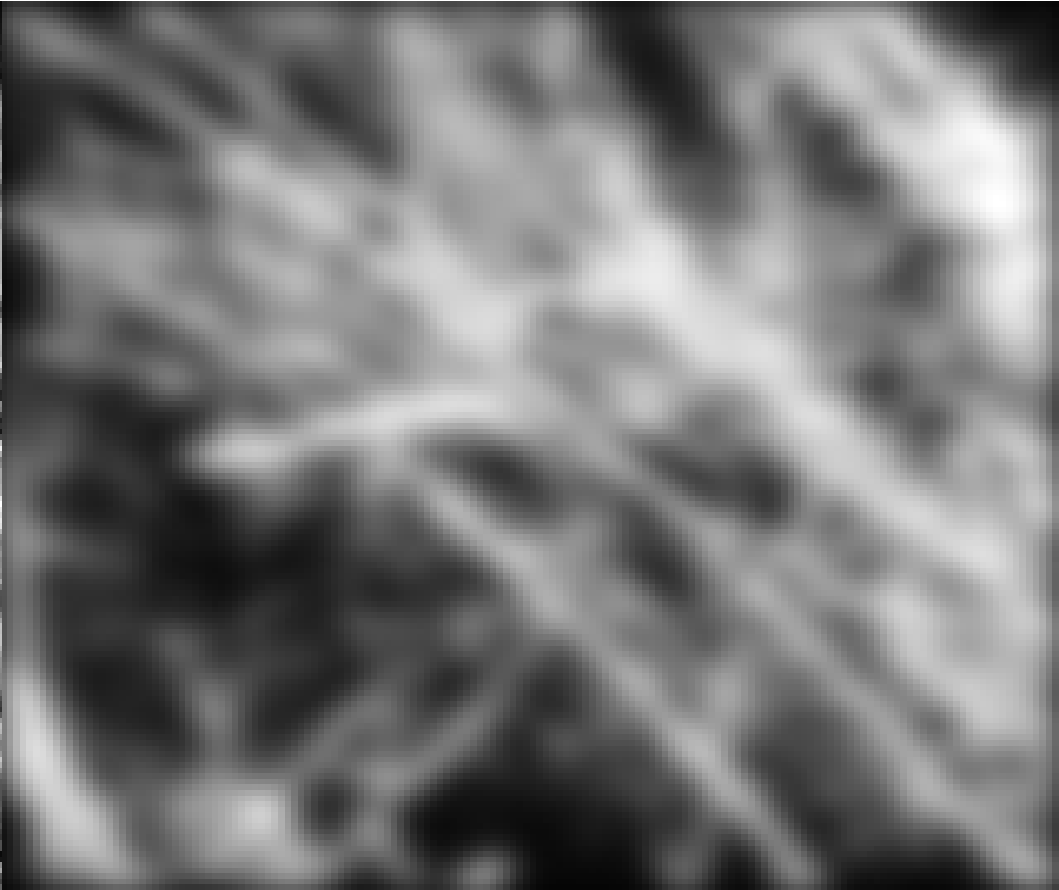


Slide credit: Robert Collins

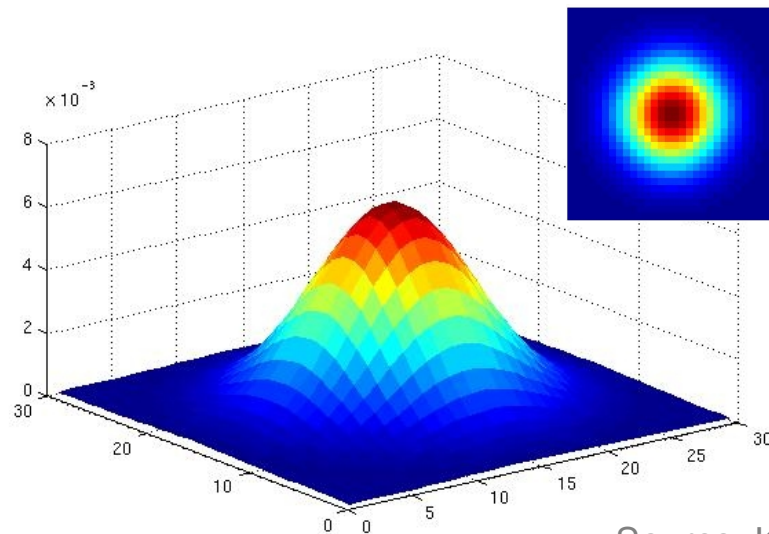
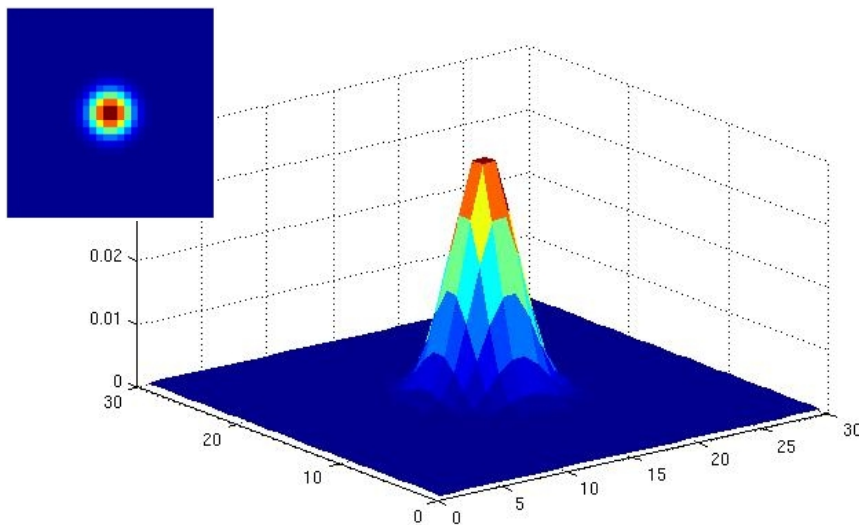
SMOOTHING: mean filtering artifacts



SMOOTHING: Gaussian



What parameters matter here?
Variance of Gaussian: determines
extent of smoothing

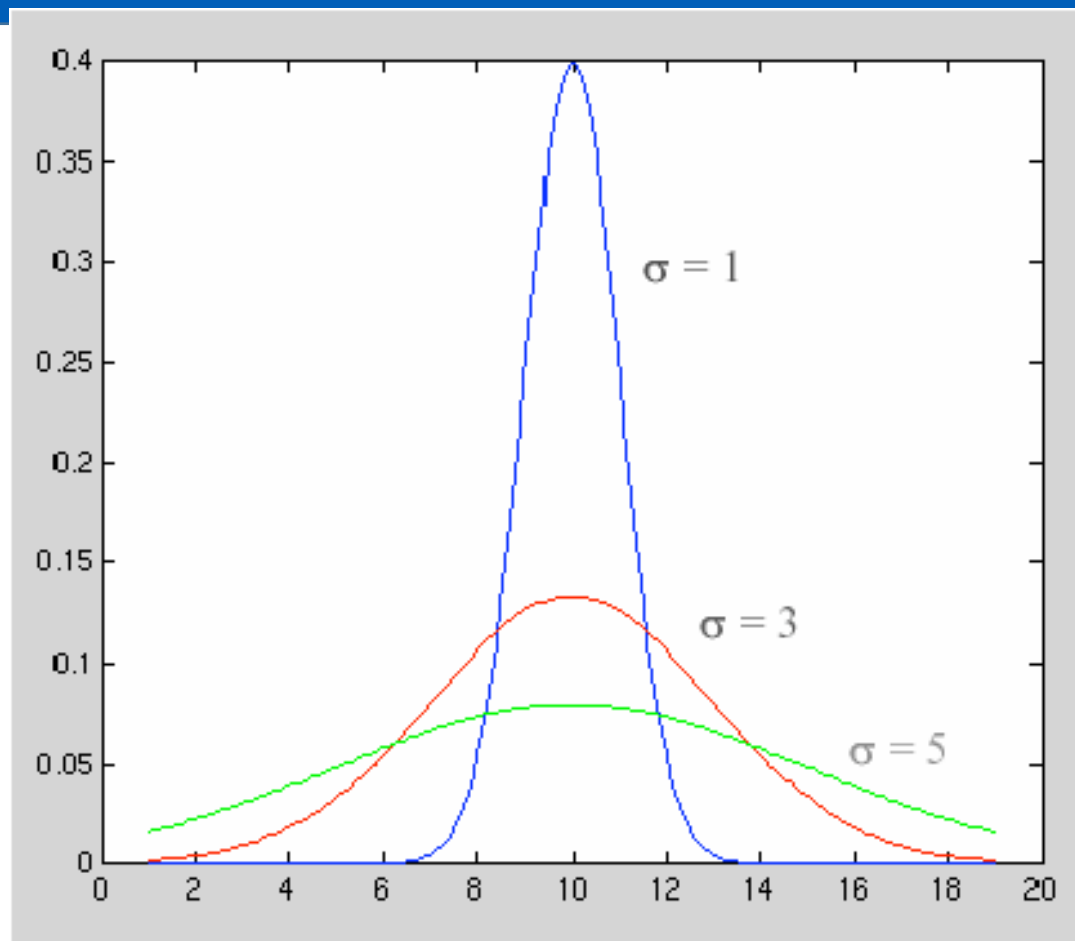


Source: K. Grauman

SMOOTHING: Gaussian

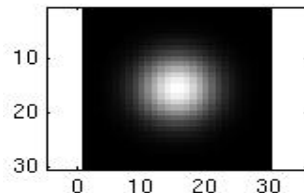
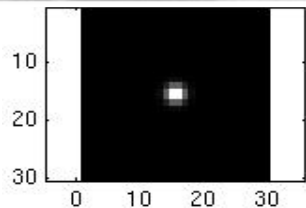
What parameters matter here?

Variance of Gaussian:
determines extent of
smoothing

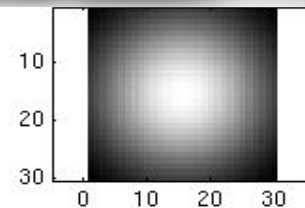


SMOOTHING: Gaussian

Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.



...



Source: K. Grauman

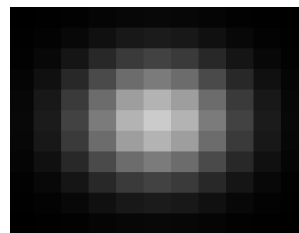
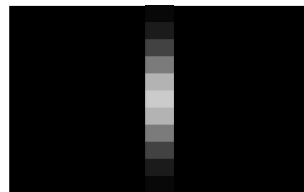
SMOOTHING: efficient Gaussian

$$\mathcal{G}_\sigma = \mathcal{G}_\sigma^x * \mathcal{G}_\sigma^y$$

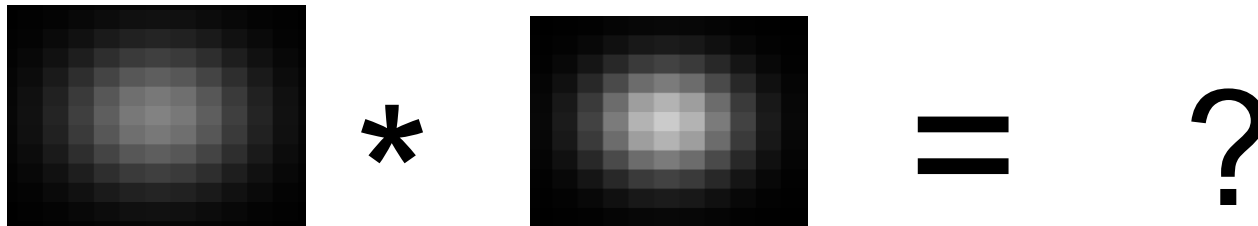
$$\mathcal{G}_\sigma^x(x, y) = \frac{1}{Z} e^{\frac{-(x^2)}{2\sigma^2}}$$

$$\mathcal{G}_\sigma^y(x, y) = \frac{1}{Z} e^{\frac{-(y^2)}{2\sigma^2}}$$

Gaussian filter is **separable**: compute in **horizontal** direction, followed by the **vertical** direction.



Faster!

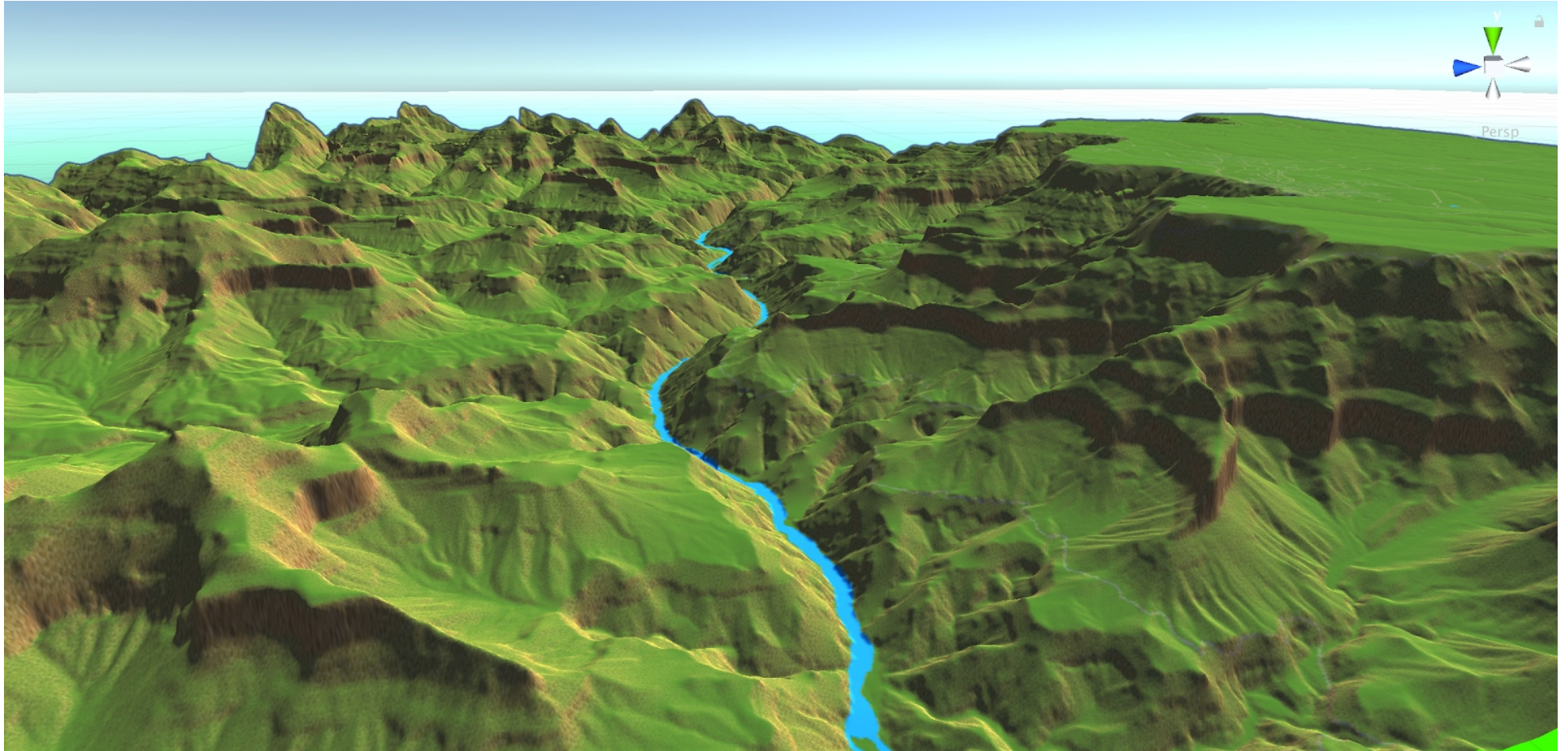


$$f * \mathcal{G}_\sigma * \mathcal{G}_{\sigma'} = f * \mathcal{G}_{\sigma''}$$

$$\sigma'' = \sqrt{\sigma^2 + \sigma'^2}$$

Cascade Gaussians: we can use smaller kernels

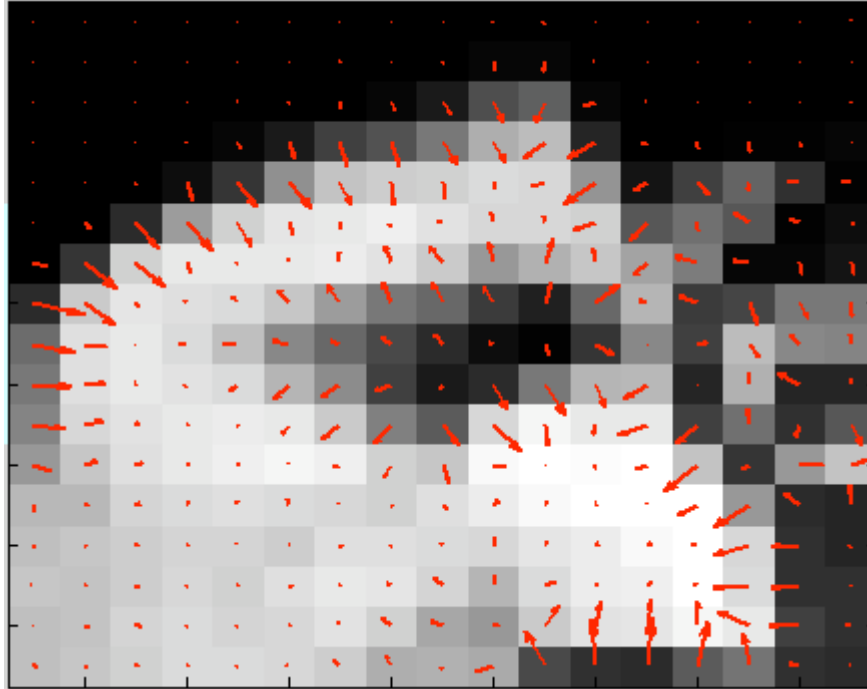
Gradient filtering



- We can see images as 2D “surfaces” where pixel “intensity” encodes elevation
- Therefore we can use concepts like:
 - Uphill/Downhill
 - Peaks/Valleys
 - Discontinuities/Steepness
- Do you remember we can see images as functions?
 - $I(x,y)$
- For each (x,y) what function can say us something about local intensity variations?
 - Gradient

$$\nabla I(x,y) = \begin{bmatrix} \frac{\partial I(x,y)}{\partial x} \\ \frac{\partial I(x,y)}{\partial y} \end{bmatrix}$$

Gradient filtering



Credits: Robert Collins

- For each pixel we have a vector
 - Magnitude
 - Direction
- Images are discrete functions
 - How we can compute numerical derivatives?

$$\frac{\partial I(x, y)}{\partial x} = \lim_{h \rightarrow 0} \frac{I(x+h, y) - I(x, y)}{h}$$

- Taylor Series Expansion can be used
- Precision of the following formula is proportional to h^2

$$\frac{\partial I(x, y)}{\partial x} \approx \frac{I(x+h, y) - I(x-h, y)}{2h}$$

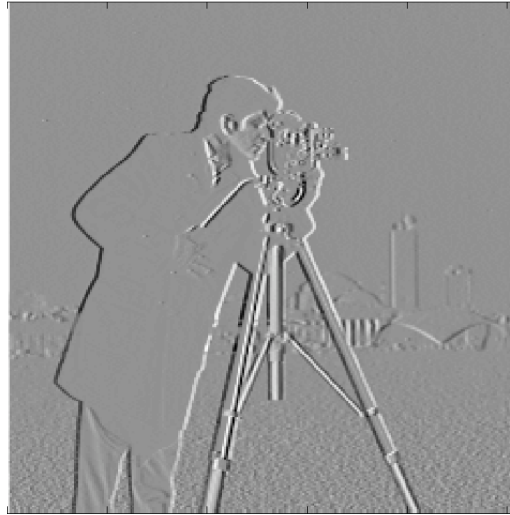
$$\frac{\partial I(x, y)}{\partial y} \approx \frac{I(x, y+h) - I(x, y-h)}{2h}$$

Gradient filtering

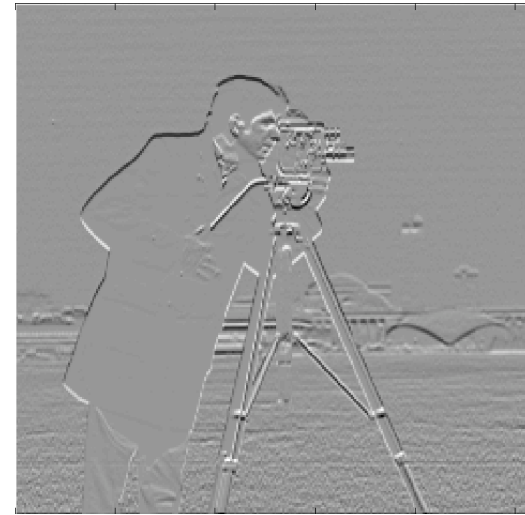
$$I(x, y)$$



$$I_x = \frac{I(x+1, y) - I(x-1, y)}{2}$$



$$I_y = \frac{I(x, y+1) - I(x, y-1)}{2}$$



Credits: Robert Collins

Gradient filtering

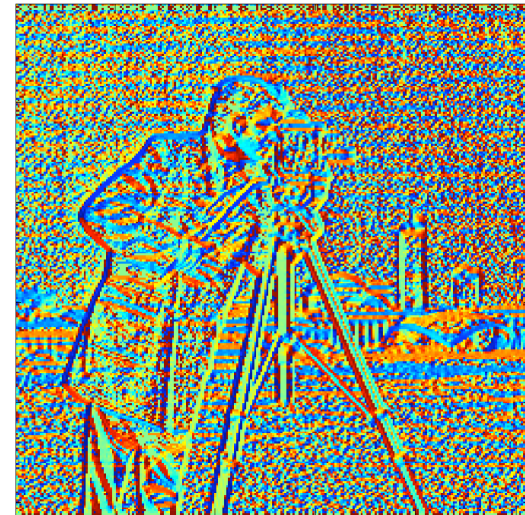
$$I(x, y)$$



$$\sqrt{I_x^2 + I_y^2}$$



$$\text{atan2}(I_y, I_x)$$



Credits: Robert Collins

- I_x and I_y can be easily computed using simple operators

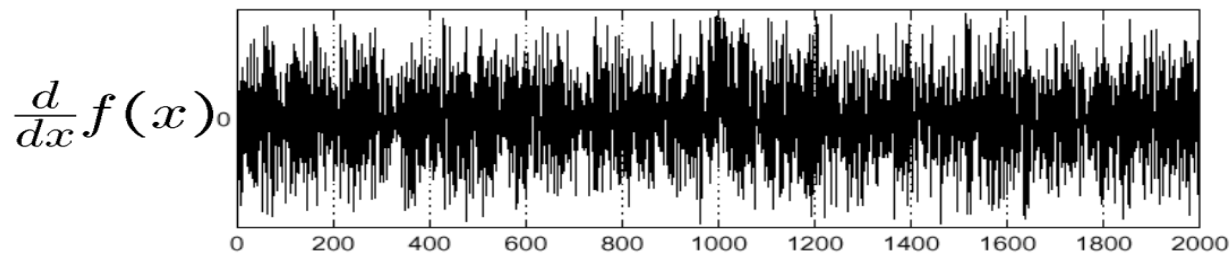
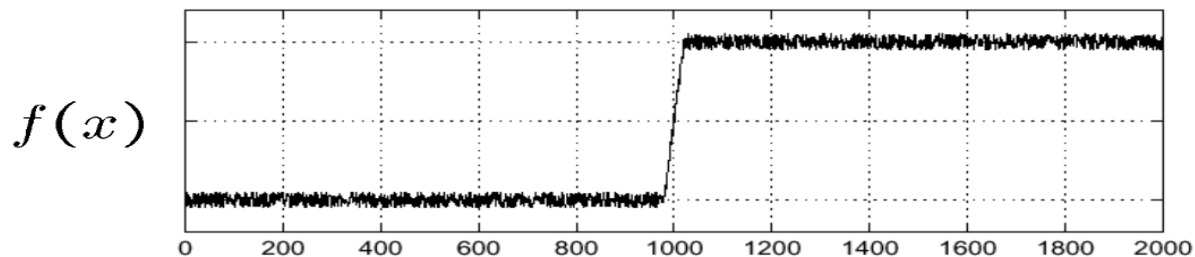
$$I_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} * I$$

$$I_x = \begin{bmatrix} +1 & 0 & -1 \end{bmatrix} * I$$

- Problem: noise

Edge filtering

Consider a single image row



Finding the “edge” is not so easy...

- Usually we want to find *strong* gradient variations (edges)
- How to remove noise? → smoothing

$$I_y = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} * ([1 \ 1 \ 1] * I) \quad I_x = [+1 \ 0 \ -1] * \left(\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * I \right)$$

- Usually we want to find *strong* gradient variations (edges)
- How to remove noise? → smoothing

$$I_y = \begin{bmatrix} +1 & 0 & -1 \\ +1 & 0 & -1 \\ +1 & 0 & -1 \end{bmatrix} * I$$

$$I_x = \begin{bmatrix} +1 & +1 & +1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} * I$$

- Prewitt

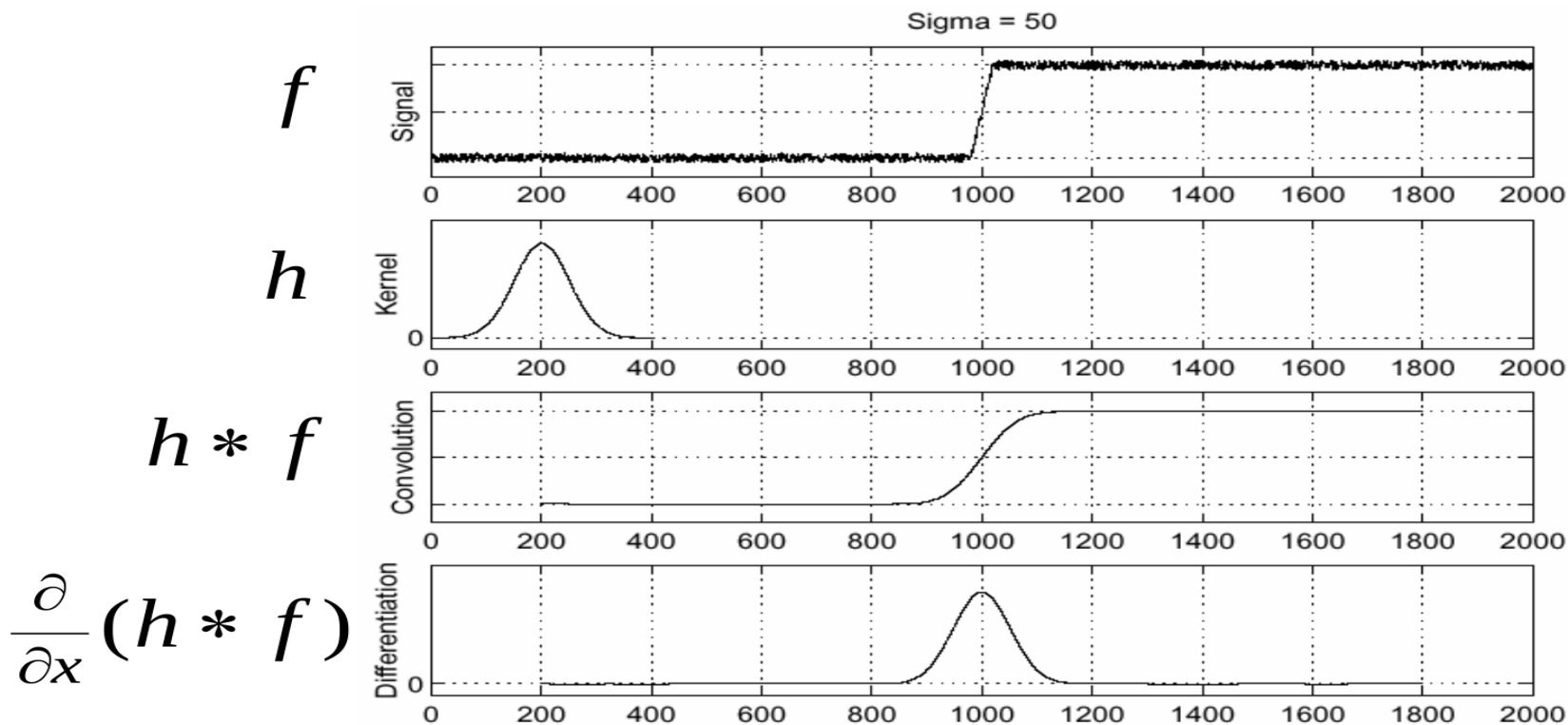
- As seen for smoothing for prewitt operators we have the same weight
- **Sobel** operators slightly improves this

$$I_y = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * I$$

$$I_x = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * I$$

- For smoothing we also saw a gaussian approach

Gradient filtering

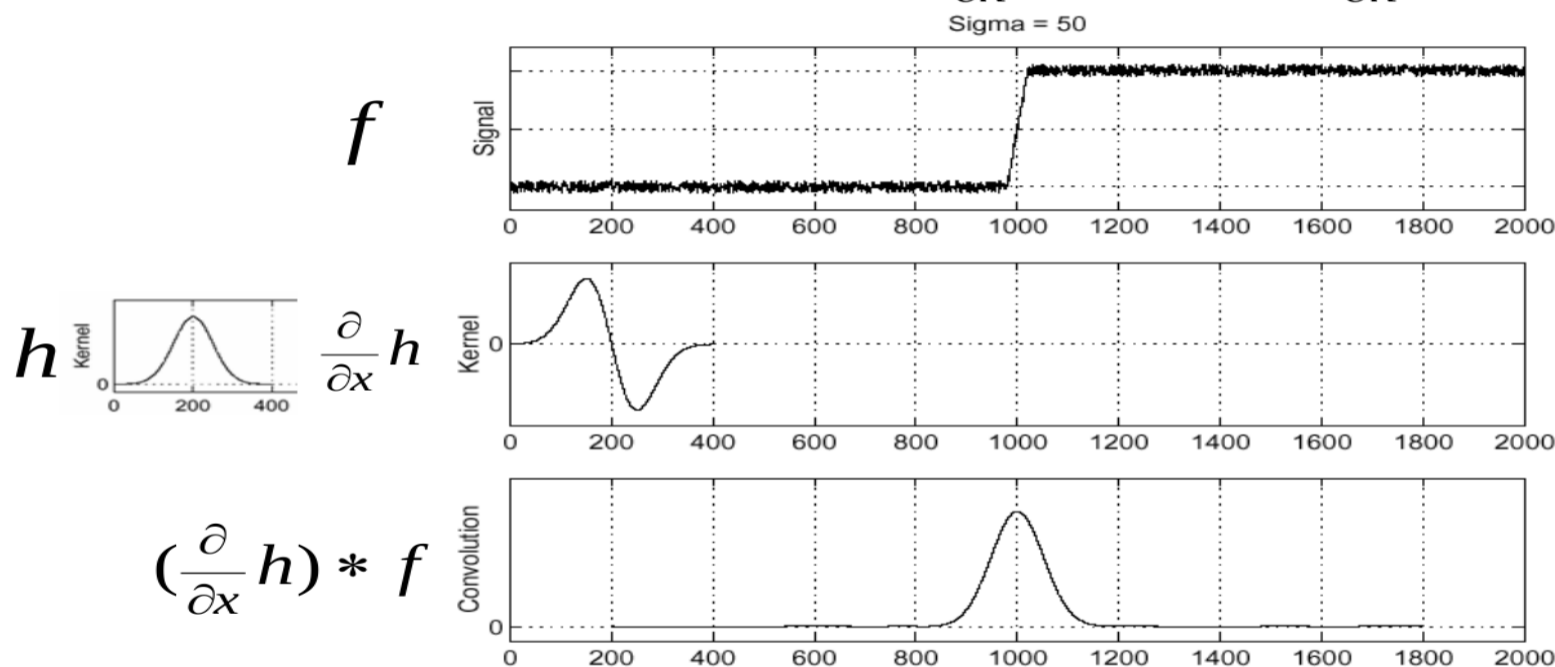


Where is the maximum gradient?

- Introducing Prewitt/Sobel we distributed the convolution
 - Namely we applied the derivative operator to smoothing one
- Derivative theorem of convolution

$$\frac{\partial}{\partial x}(f * g) = \left(\frac{\partial}{\partial x} f \right) * g = f * \left(\frac{\partial}{\partial x} g \right)$$

- This saves us one operation: $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x} h) * f$



And if we want to find local maxima?

- 1st derivative filters
- Where is the edge?
 - Magnitude peaks

$$\sqrt{\frac{\partial}{\partial x} I(x, y)}$$

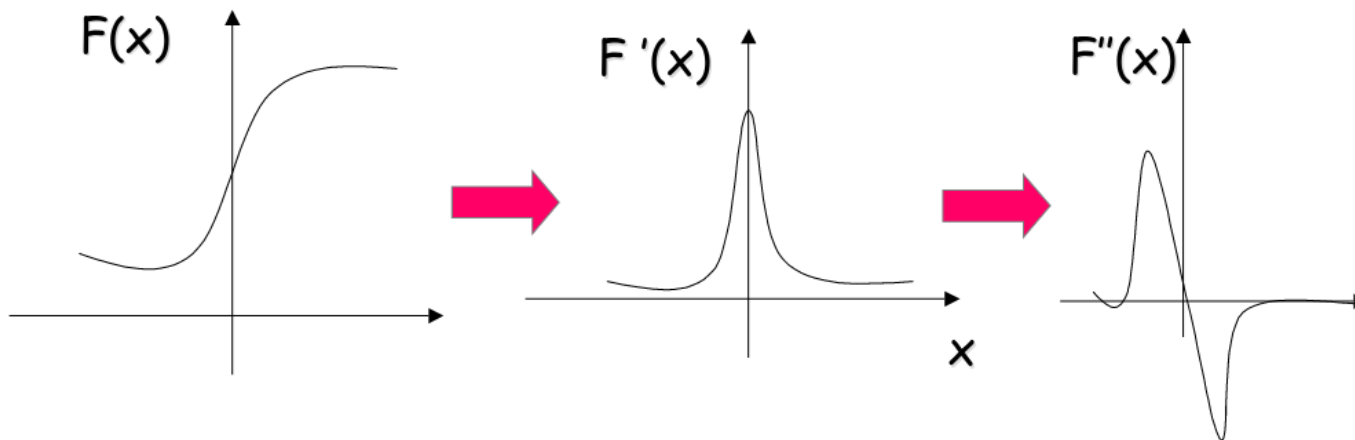


- 1st derivative filters
- Where is the edge?
 - Magnitude peaks

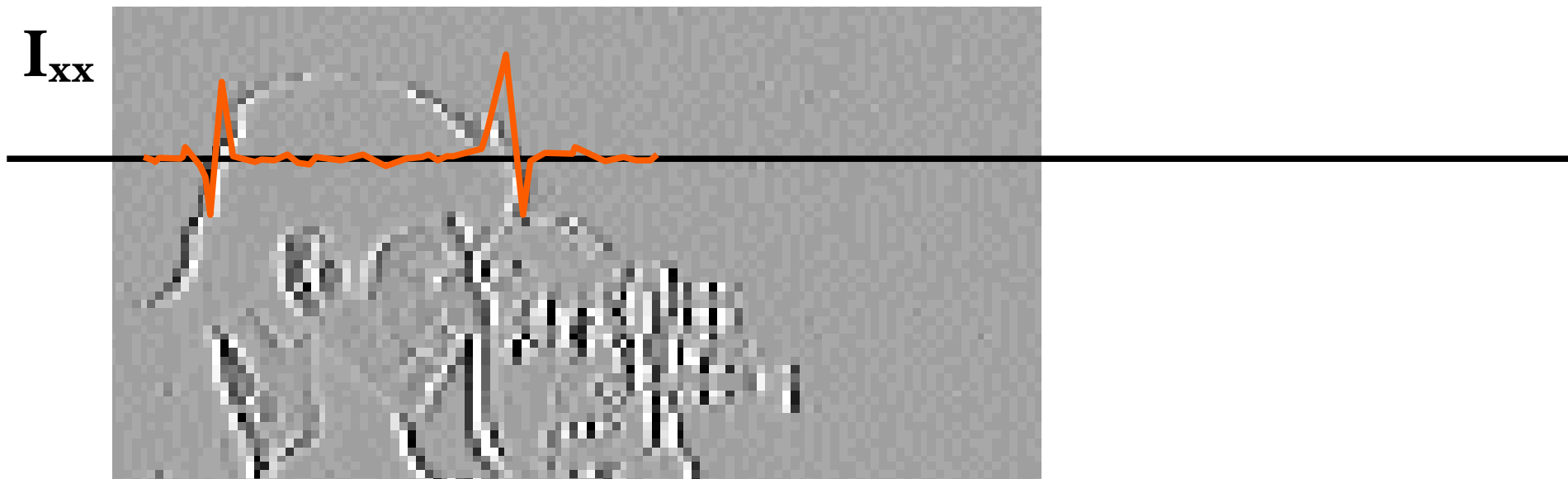
$$\sqrt{I_x^2 + I_y^2} > Th$$



- Using 1st derivative filters maxima detect edges
 - Not simple to exactly find them
- What about 2nd derivative filters?



- Pro: easy localization of edges
- Cons: 2nd derivative is 0 also when 1st one is 0



Gradient Filtering: Laplacian operator

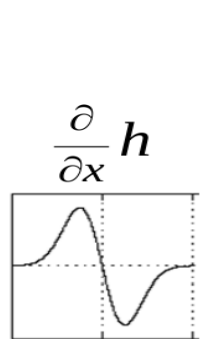
- Laplacian operator ∇^2 can be used to combine I_{xx} and I_{yy}

$$\nabla^2 I(x, y) = I_{xx}(x, y) + I_{yy}(x, y) \approx \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} * I$$

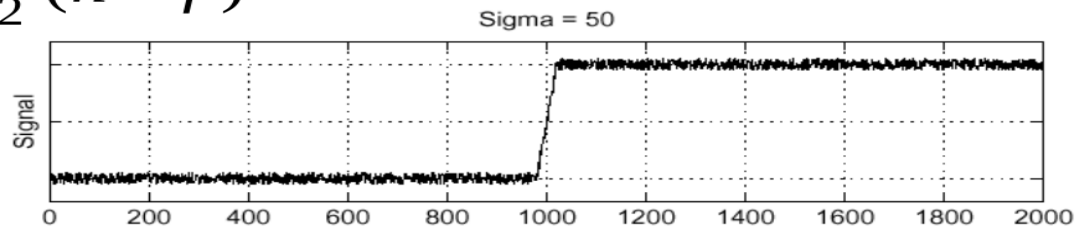


Gradient filtering

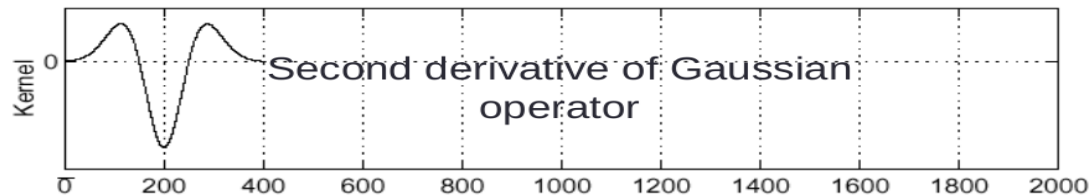
- Consider $\frac{\partial^2}{\partial x^2} (h * f)$



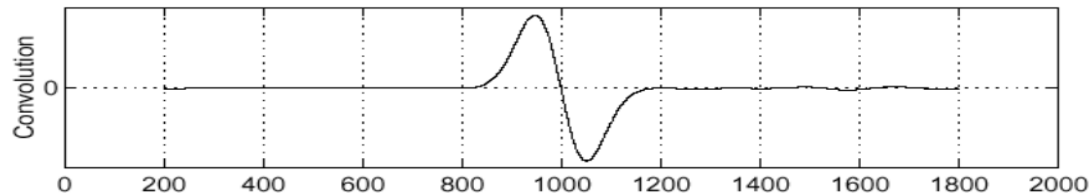
f



$\frac{\partial^2}{\partial x^2} h$

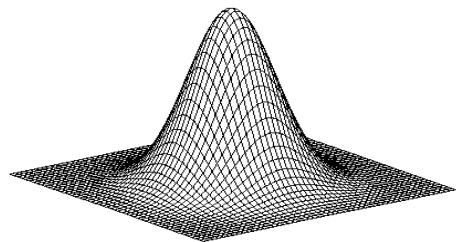


$\frac{\partial^2}{\partial x^2} (h * f)$



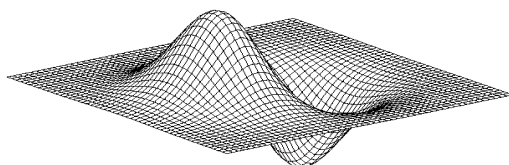
The maxima is where we have a zero crossing

Source: A. Bobick



Gaussian

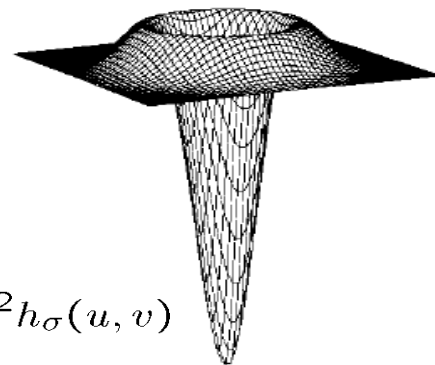
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

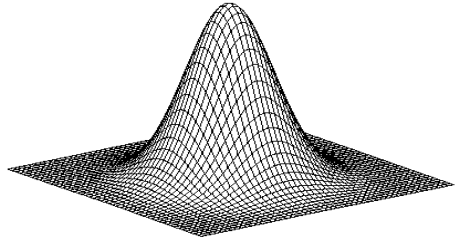
Laplacian of Gaussian
or LoG filter



$$\nabla^2 h_{\sigma}(u, v)$$

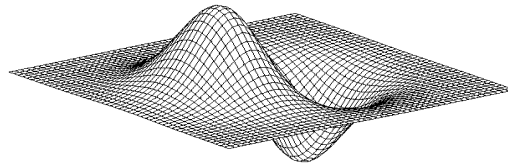
∇^2 is the Laplacian operator (sum of 2nd derivatives):

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$



Gaussian

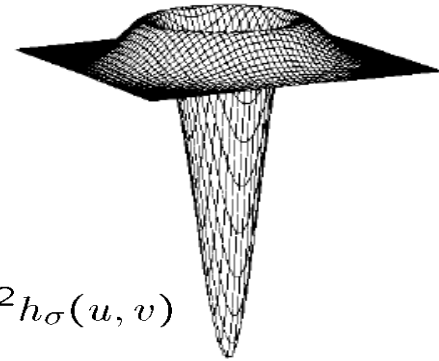
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

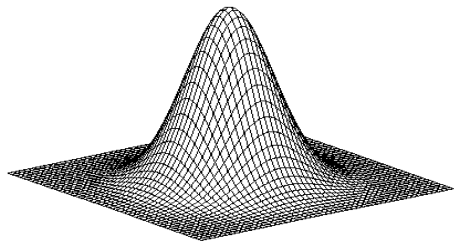
Laplacian of Gaussian
or LoG filter



$$\nabla^2 h_{\sigma}(u, v)$$

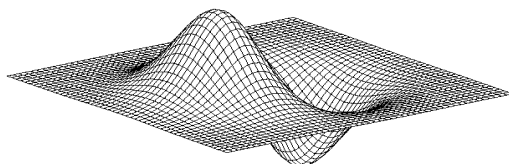
$$\begin{aligned} &\text{Laplacian(Gaussian * f)} \\ &= \\ &\text{LoG * f} \end{aligned}$$

Gradient filtering



Gaussian

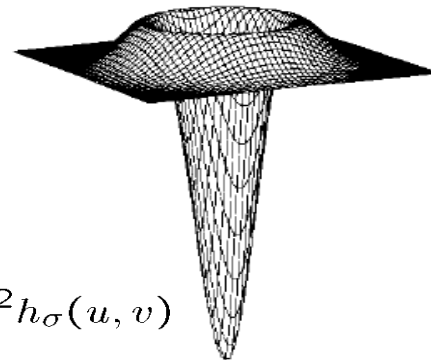
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian
or LoG filter



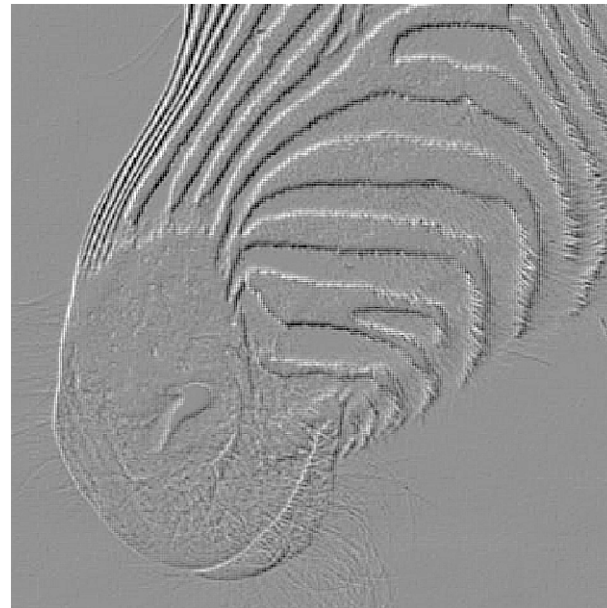
$$\nabla^2 h_{\sigma}(u, v)$$

$$\nabla^2 G(x, y; \sigma) = \left(\frac{x^2 + y^2}{\sigma^4} - \frac{2}{\sigma^2} \right) G(x, y; \sigma)$$

Often approximated
by

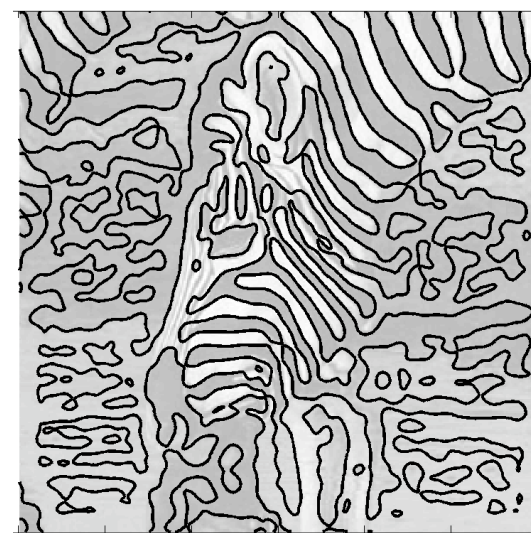
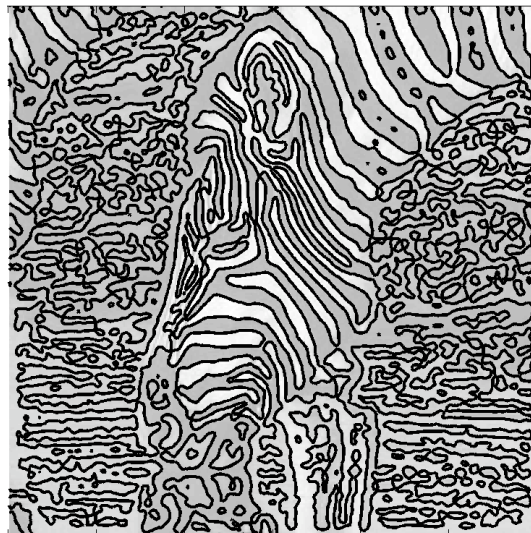
0	1	0
1	-4	1
0	1	0

- Band pass filter
 - Suppress low frequencies (2^{nd} derivative) but also high ones (gaussian)



LoG Example: zero crossing

- Used as “edge” detector, extract “raw” zero crossings:
 - Different sigmas (2 & 4)
 - Like a “equal elevation” on a map



- We now should have understood basic convolution principles
- We can add further complexity
 - Border issues
 - Padding
 - Stride

- How we deal with border/corner pixels in a convolution?
 - Actually the same issue for other operations that involve a mask/kernel
- Basically two approaches
 - Result of the processing is smaller than original image
 - Same size with border/corner pixels with *unknown* value

- **Padding** is a technique to “enlarge” images adding a given amount of pixels to each image border
- The most common approach is the **zero padding**
 - Added pixels are set to 0

- Padding is often use before a convolution to obtain same size result
- The “extra” pixels can be set using different policies
 - Again: zero padding is the most used



black



fixed



periodic



reflected

- For a 4x4 image

20	128	60	100
50	66	72	20
73	98	65	89
20	170	190	200

- Zero padding 1 $\rightarrow$ output size is **6x6**

0	0	0	0	0	0
0	20	128	60	100	0
0	50	66	72	20	0
0	73	98	65	89	0
0	20	170	190	200	0
0	0	0	0	0	0

- When we use a 3x3 kernel we obtain a 4x4 resulting image

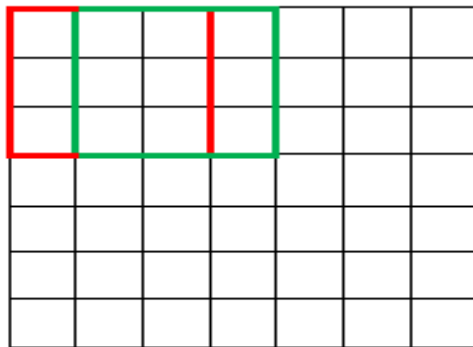
- Zero padding 2 $\rightarrow$ output size is **8x8**

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	20	128	60	100	0	0
0	0	50	66	72	20	0	0
0	0	73	98	65	89	0	0
0	0	20	170	190	200	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

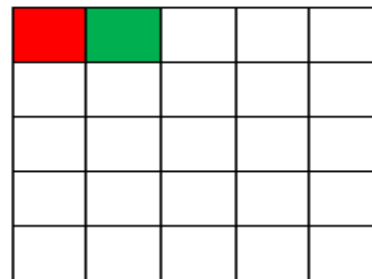
- The stride is a parameter used to set kernel movements on the image
- Since now we assumed that kernel is horizontally and vertically moved on each row/column $\rightarrow$ stride is 1
- We can move a kernel skipping some pixel $\rightarrow$ stride > 1
- A stride > 1 leads to subsampling images!
- Moreover we can have different strides for rows/columns

- Stride = 1 $\rightarrow$ “normal convolution”. Kernel is moved on every pixel (vertically & horizontally)
- I.e for 7x7 image, a 3x3 kernel and a stride value as 1 we will obtain a 5x5 image

7 x 7 Input Volume

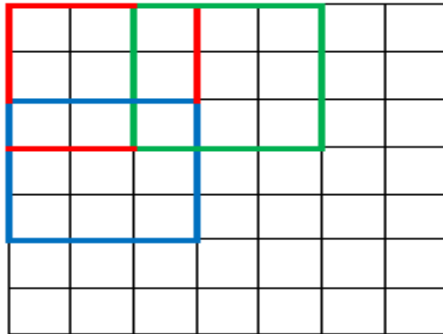


5 x 5 Output Volume



- Stride = 2 $\rightarrow$ kernel is moved skipping 1 column for each horizontal move and 1 row for each vertical move
- I.e for 7x7 image, a 3x3 kernel and a stride value as 1 we will obtain a **3x3 image**

7 x 7 Input Volume



3 x 3 Output Volume



- Given a image featuring n columns
- Using a kernel with f columns, with padding p and stride s , the number of columns of the output image is:

$$n_{new} = \left\lfloor \frac{n + 2 * p - f}{s} + 1 \right\rfloor$$

- The result must be the largest integer value equal or smaller than n_{new}
- The same formula can be used for computing the new number of rows



UNIVERSITÀ DI PARMA

Image Filtering Local & NON Linear operations

- Since now we illustrate the so-called **linear filters**
 - The result is a linear sum of neighborhood values
 - Linear filters can easily be combined:
 - The result of the processing of the sum of 2 signals can be obtained as the sum of the processing of the 2 signals

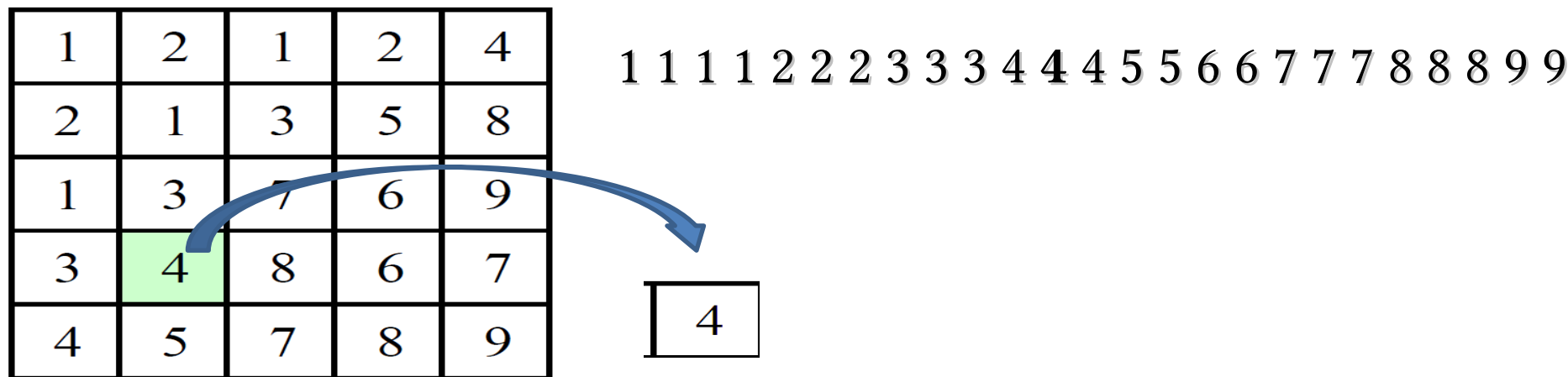
$$S(\alpha * f[x, y] + \beta * g[x, y]) = \alpha * S(f[x, y]) + \beta * S(g[x, y]) \text{ for all } \alpha, \beta \in \mathbb{R}$$

- Linear filters can be easily combined and can also be analyzed in the frequency domain (Fourier transform) but they are not always the most efficient solution

- Median pooling
- Max/Min pooling
- *Average pooling*
- Bilateral filter

Median pooling

- For each neighborhood we select the median value



- Is a smoothing operation
- Can cope with salt & pepper noise
- Sorting needed → computationally expensive

Median pooling



Shot noise



5x5 average



5x5 median

Max/Min pooling

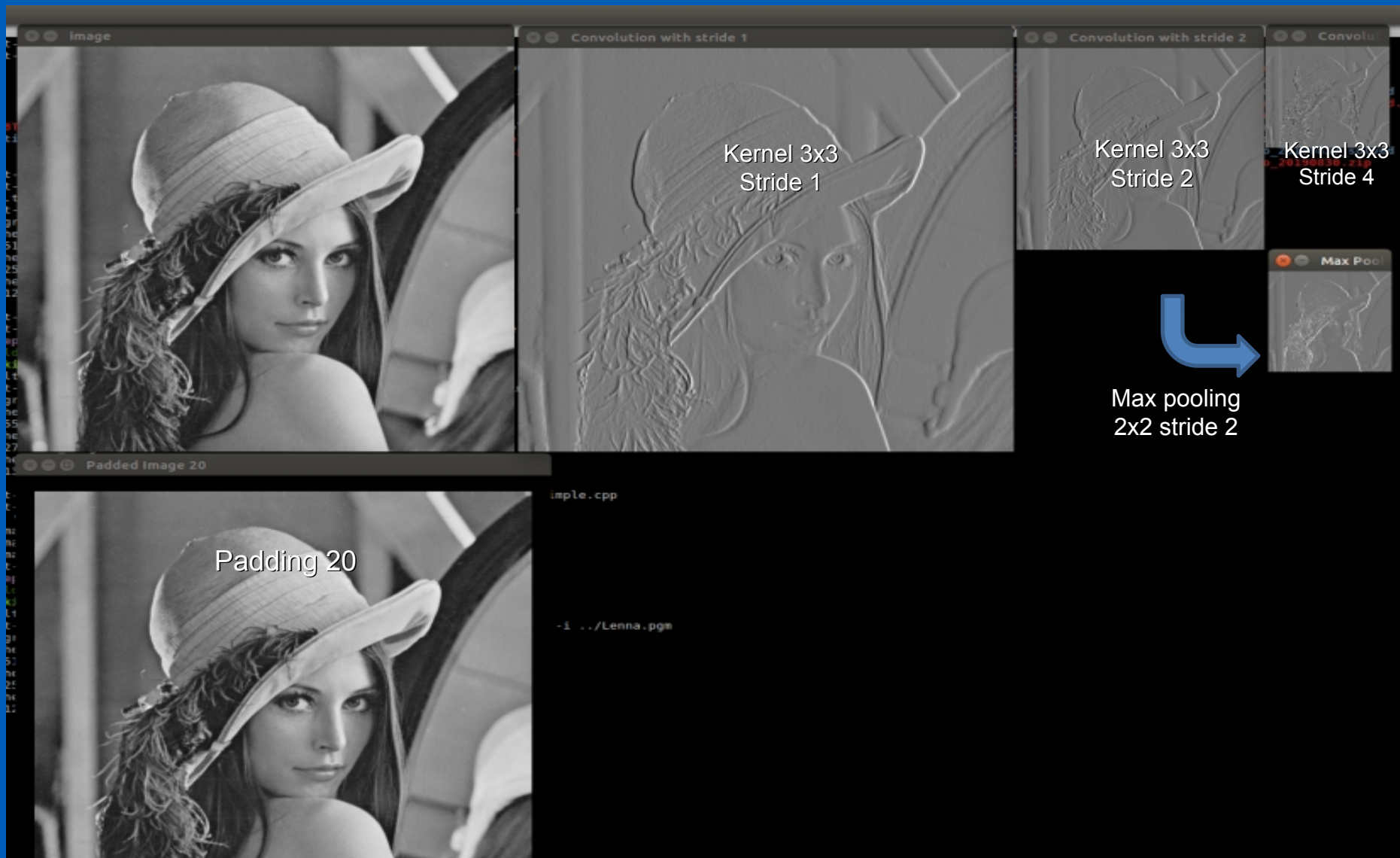
- Select the largest/smallest value in the neighborhood

20	128	60	100
50	66	72	20
73	98	65	89
20	170	190	200

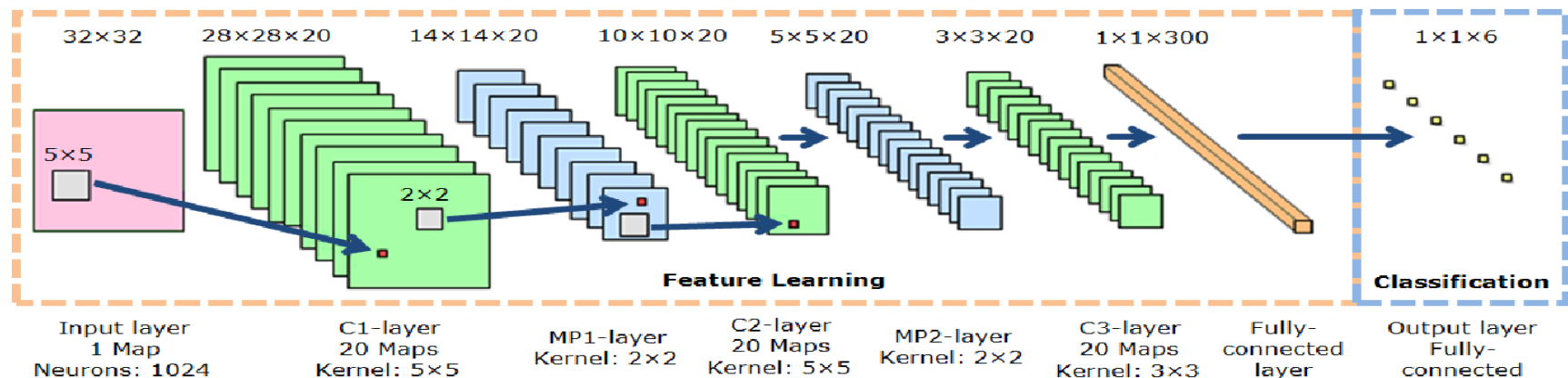
128	128	100
98	98	89
170	190	200

- Can be used to selectively remove salt & pepper noise

- We already saw this guy!
- Box filter (linear)



- Convoluzioni con stride e max pooling sono gli elementi base di una **Convolutional Neural Network (CNN)**:





UNIVERSITÀ DI PARMA

Image Filtering

Integral Image

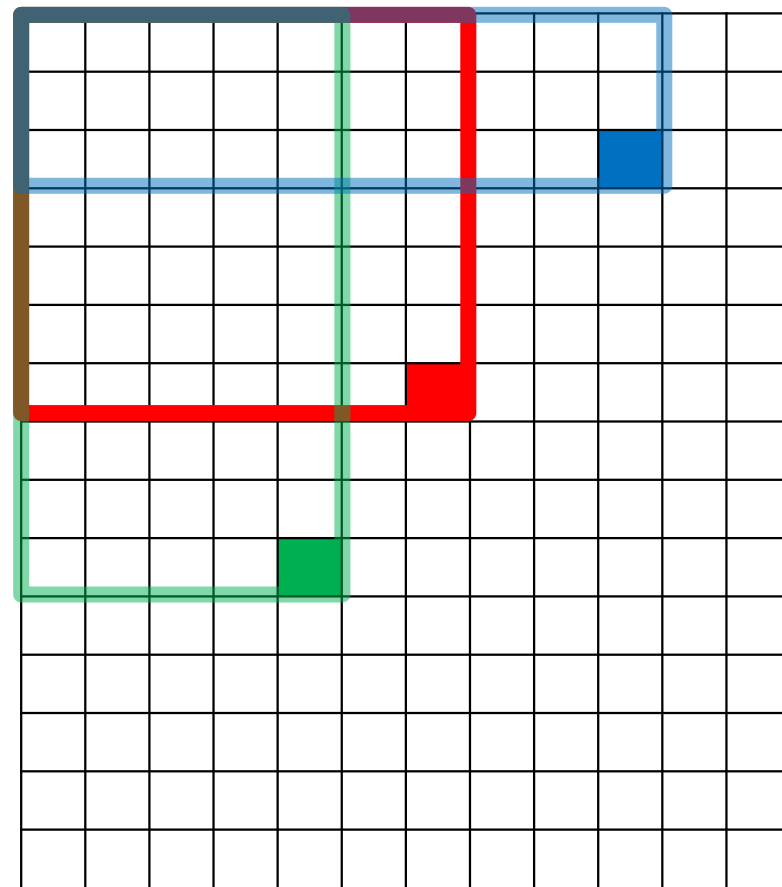
- Often convolutions imply to repeatedly compute the sum of a region.

$$S(i, j) = \sum_{k=0}^i \sum_{l=0}^j F(k, l)$$

- This is an expensive operation $\rightarrow O(n^2)$
- The **integral image** can lead to $O(n)$
- Also known as **summed area table**

243	239	240	225	206	185	188	218	211	206	216	225
242	239	218	110	67	31	34	152	213	206	208	221
243	242	123	58	94	82	132	77	108	208	208	215
235	217	115	212	243	236	247	139	91	209	208	211
233	208	131	222	219	226	196	114	74	208	213	214
232	217	131	116	77	150	69	56	52	201	228	223
232	232	182	186	184	179	159	123	93	232	235	235
232	236	201	154	216	133	129	81	175	252	241	240
235	238	230	128	172	138	65	63	234	249	241	245
237	236	247	143	59	78	10	94	255	248	247	251
234	237	245	193	55	33	115	144	213	255	253	251
248	245	161	128	149	109	138	65	47	156	239	255
190	107	39	102	94	73	114	58	17	7	51	137
23	32	33	148	168	203	179	43	27	17	12	8
17	26	12	160	255	255	109	22	26	19	35	24

- Each pixel (x,y) contains the sum of other pixels contained in the rectangle $(0,0)-(x,y)$



Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4				

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7			

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14		

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6				

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14			

- A fast way to compute the summed area table is:
 - Initialize integral image I using same values of original image
 - Start from (0,0) and move row by row
 - Update each “cell” as
 - $I(x, y) = I(x, y) + I(x-1, y) + I(x, y-1) - I(x-1, y-1)$

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	50
11	25	43	68	74
12	27	49	77	85
12	29	54	86	95

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	50
11	25	43	68	74
12	27	49	77	85
12	29	54	86	95

Let's try to compute the sum of that area $7+9+1+4+7+2+4+3+2 = 39$

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85 _A
12	29	54	86	95

Consider the same area on the Integral Image and cells A, B, C, D

Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85 _A
12	29	54	86	95

The result is $A+B-C-D \rightarrow 85 + 7 - 26 - 27 = 39$

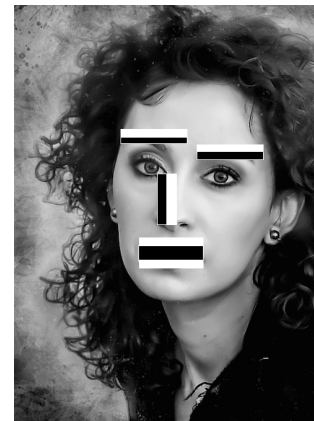
Integral Image

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85 _A
12	29	54	86	95

The result is $A+B-C-D \rightarrow 85 + 7 - 26 - 27 = 39$

- Examples of usage
 - Filter box: given a $n \times n$ kernel on a $M \times N$ image
 - No integral image, separable kernels $\rightarrow O(M \times N \times 2n)$
 - Computing integral image $\rightarrow O(M \times N \times 2)$
 - Haar like features
 - Binary patterns used for face detection





UNIVERSITÀ DI PARMA

Image Filtering

IMAGE FILTERING



- What is an Image?
- Image Filtering
- Local Operations
 - Image Convolution
 - Linear filters
 - Smoothing
 - Gradient
 - Padding
 - Stride
 - Non linear filters
 - Bilateral filtering
 - Integral Image

What are images?



Prendiamo ad esempio una immagine a caso

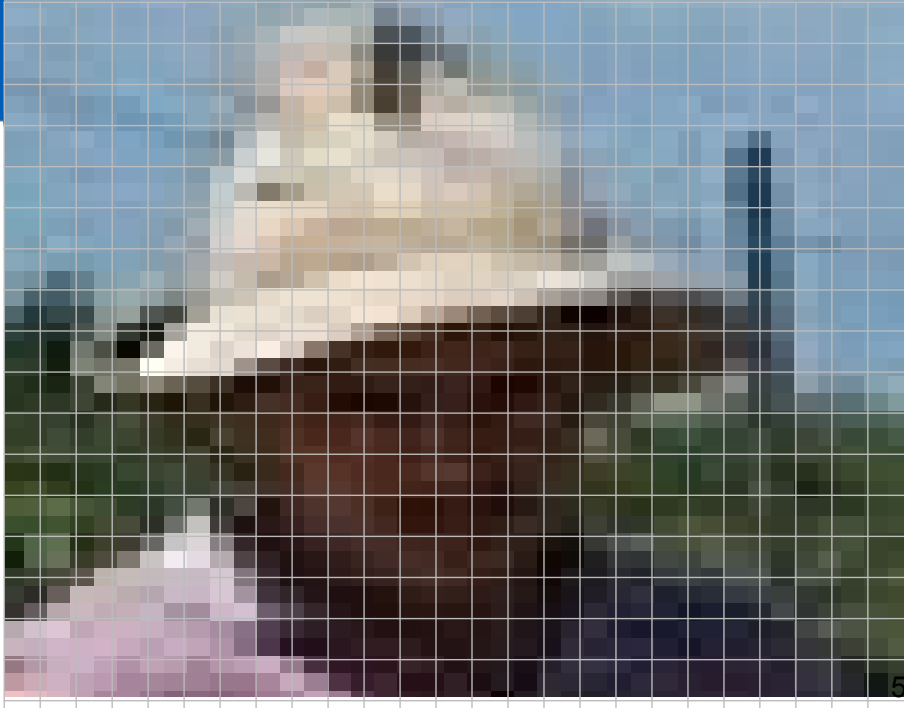
What are images?



$$P = f(x, y)$$

$$f : \mathbb{R}^2 \Rightarrow \mathbb{R}$$

La posso vedere come una funzione 2D. Date le coordinate di riga e colonna (x,y) ottengo un valore. Che valore? Lo vedremo nel dettaglio successivamente ma nella forma più generale possibile è un numero reale (sì, ci possono essere anche valori non interi). Ma in realtà anche le coordinate di riga e colonna nella forma più generale potrebbero essere non interi e quindi ho una funzione da $\mathbb{R}^2$ a $\mathbb{R}$.



UNIVERSITÀ
DI PARMA



$$P = f(x, y)$$

$$f : \mathbb{R}^2 \Rightarrow \mathbb{R}$$

1. Signal is **sampled** to obtain a function (r, c)
2. Result is also **quantized** given we need a discrete function
3. One or more **channels** for each pixel

$R(r, c), G(r, c), B(r, c)$

Luminance(r, c)

Va comunque detto che la forma generale quasi mai è di interesse, per cui si quantizza e la mia funzione fornisce numeri interi. E in generale coordinate non intere sono di interesse in casi relativamente isolati.

Insomma si quantizza tutto.

Inoltre posso per ciascun punto di una immagine (**pixel**) avere più canali e quindi in realtà più funzioni.

Ad esempio per immagini a colori è abbastanza comune avere 3 canali, uno per ciascuna componente del colore (R rosso, G verde, B blu). Mentre per le immagini a toni di grigio tipicamente ne ho uno solo.

Ma esistono comunque situazioni anche più variegate (ci torneremo quando parleremo di formati immagine).

$$F(\text{img1}) = \text{img2}$$

Ora iniziamo a parlare di elaborazione immagini (non è di fatto visione artificiale).

La possiamo definire come applicazione di una funzione spesso del tipo $\mathbb{R}^2 \rightarrow \mathbb{R}^2$

In questa carrellata si vedono diversi esempi.

Quasi sempre il risultato dell'elaborazione di una immagine è una seconda immagine ma non necessariamente.



$$F(\text{img}) = \text{img}$$

0.1
0
0.8
0.9
0.9
0.9
0.2
0.4
0.3
0.6
0
0
0.1
0.5
0.9
0.9
0.2
0.4
0.3
0.6
0
0
0.1
0.9
0.9
0.2
0.4
0.3



$$F\left(\text{img}_1, \text{img}_2\right) = \text{img}_3$$

The equation illustrates an image filtering operation. The first image (img₁) shows a person wearing a straw hat. The second image (img₂) shows a horse's head. The result (img₃) is a composite image where the horse's head is superimposed onto the person's face, demonstrating a filtering or masking operation.

- Filters
 - Mathematical functions that operate on the pixels
 - Often Image $\rightarrow$ Image
- Different classes
 - **Point operations:** result depends only on each pixel value
 - **Local operations:** result depends on each pixel + neighborhood values
 - **Global operations:** result depends on the values of all image pixels

Quindi l'elaborazione immagine è una funzione che opera sui pixel e spesso produce una seconda immagine come risultato.

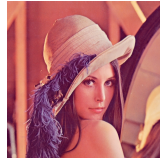
Si possono individuare 3 tipologie:

- Operazioni puntuali: la funzione applicata lavora sui singoli pixel, quindi il risultato dipende solo dal pixel in esame (esempi: modifica contrasto, luminosità...)
- Operazioni locali: il risultato dipende da un "intorno" più o meno ampio di ciascun pixel. Quindi prendo in esame un'area intorno al pixel in esame (esempio, riduzione rumore)
- Operazioni globali: la funzione di trasformazione prende in ingresso tutta l'immagine (esempio, trasformata di Fourier)

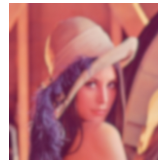
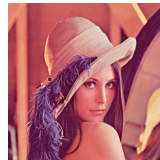
Image Filtering Examples



- Point operations:
 - Color to gray levels



- Local operations:
 - Running average (blur)



- Global operations:
 - Fourier transform

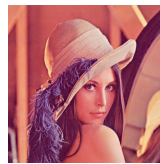


IMAGE FILTERING
LOCAL & LINEAR OPERATIONS
CONVLUTION

$$F(\text{Image with neighborhood dots}) = \text{Result Image}$$

- Result depends only on
 - Pixel value
 - Pixel Neighborhood

Come accennato la funzione di elaborazione dipende da un'area attorno al pixel in esame (vicinato).



- We already said that an image can be seen as a matrix
 - Each matrix element corresponds to a image point better known as **pixel**
 - Each pixel feature one or more values that allow to define its content
 - Color, luminance, transparency...
 - Not necessarily integer numbers...
- For following slides, we are going to deal with grey level only images
 - Each pixel must encode luminance only
 - Only 1 integer non negative number for each pixel
 - The higher the number, the brighter the pixel
 - 0 $\rightarrow$ black

- Image convolution is the main operation used for local image filtering
 - From $f[.,.] \rightarrow h[.,.]$
- $g[k,l]$ is the **Kernel** or Convolution Matrix and represents the pixel neighborhood weight
 - Kernel size

- **Linear operation**

- Symbol: *

- $h=g*f$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

Limitiamoci inizialmente a operazioni lineari

Indicando con $f[]$ la matrice immagine in ingresso e con $h[]$ l'immagine risultante, l'elaborazione sfrutta tipicamente una matrice di convoluzione $g[]$ o kernel con cui si pesano i pixel in ingresso.

Il kernel è una matrice di dimensioni significativamente minori di quelle dell'immagine da elaborare.

All'atto pratico, e come si vede nelle slide successive, io sovrappongo $g[]$ a $f[]$ centrandolo su ciascun pixel di $f[]$. Moltiplico i valori "sovrapposti" di $g[]$ e $f[]$ e li sommo tra di loro ottenendo il valore del corrispondente pixel di $h[]$. La formula in esempio spiega come calcolare il valore di h alle coordinate (m,l)

Local Operations

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

1	1	1
1	1	1
1	1	1

$\frac{1}{9}$

$g[.,.]$

Credit: S. Seitz

In questo caso il kernel è una matrice 3x3 composta da valori tutti pari a 1/9 (per semplicità uso un moltiplicatore esterno da 1/9 e una matrice di 1)

UNIVERSITÀ
DI PARMA

[illegible][illegible]

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Credit: S. Seitz

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20						

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

1	1	1
1	1	1
1	1	1

$\frac{1}{9}$

$g[.,.]$

Credit: S. Seitz

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30					

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

1	1	1
1	1	1
1	1	1

$\frac{1}{9}$

$g[.,.]$

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30				

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

1	1	1
1	1	1
1	1	1

$\frac{1}{9}$

$g[.,.]$

UNIVERSITÀ
DI PARMA

$$f[.,.]$$
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$
$$\frac{1}{9}$$
$$\mathbf{g}[\cdot, \cdot]$$

Credit: S. Seitz

Credit: S. Seitz

Local Operations



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[.,.]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

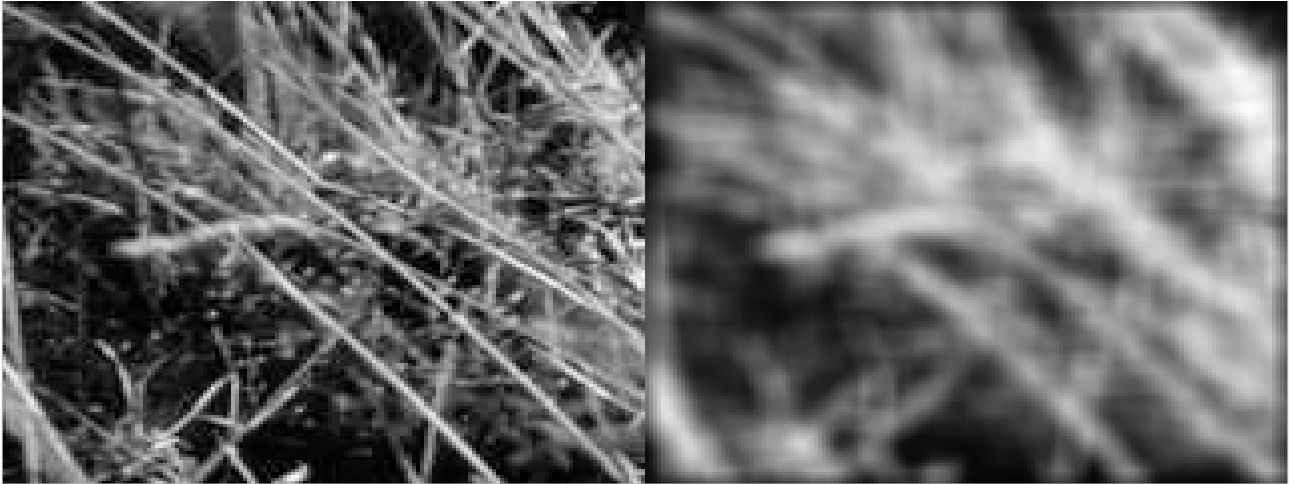
$g[.,.]$

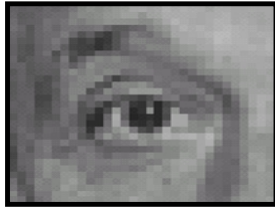
- What is the result of such filtering?
 - Neighborhood average
 - Low pass filter
- Basically sharp transitions are reduced
 - blurring
- **Mean/Box filter smoothing**

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

Il box filter è uno dei piú usati (e semplici)





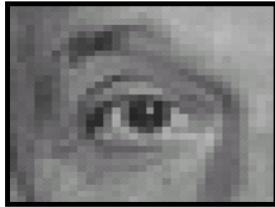
Original

0	0	0
0	1	0
0	0	0

?

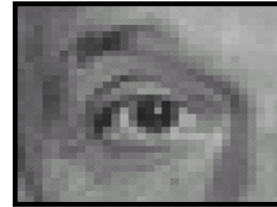
Slide credit: David Lowe (UBC)

È un esempio non molto sensato ma solo per far capire il meccanismo



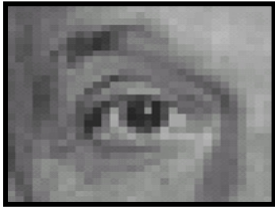
Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Slide credit: David Lowe (UBC)

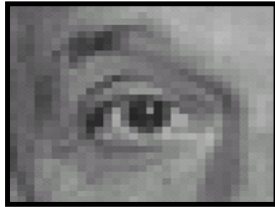


Original

0	0	0
0	0	1
0	0	0

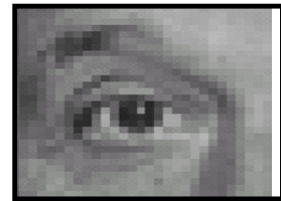
?

Slide credit: David Lowe (UBC)



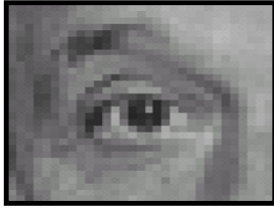
Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Slide credit: David Lowe (UBC)



Original

0	0	0
0	2	0
0	0	0

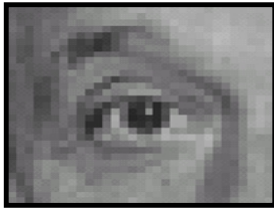
-

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

Slide credit: David Lowe (UBC)



Original

0	0	0
0	2	0
0	0	0

■

$\frac{1}{9}$

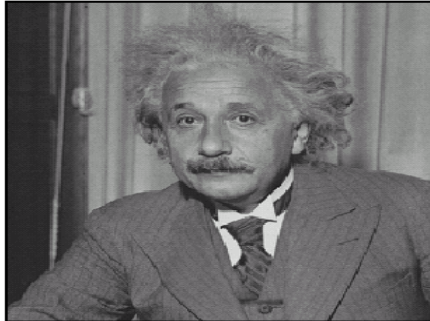
1	1	1
1	1	1
1	1	1



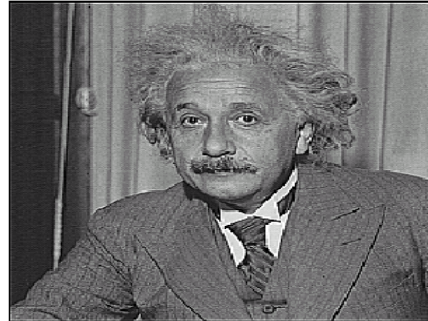
Sharpening filter

- Accentuates differences with local average
- High pass filtering

Slide credit: David Lowe (UBC)

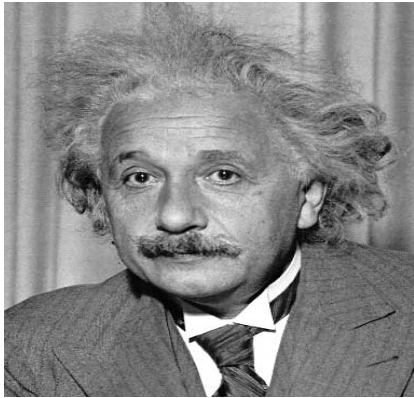


before



after

Slide credit: David Lowe (UBC)



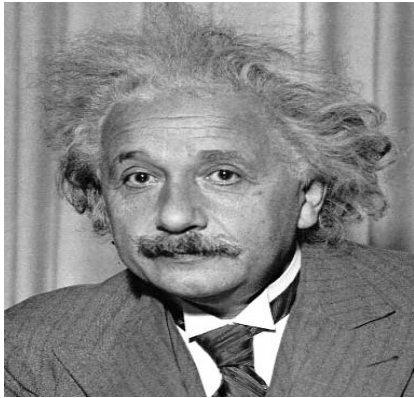
1	0	-1
2	0	-2
1	0	-1

Vertical
Sobel



**Vertical Edge
(absolute value)**

Slide credit: David Lowe (UBC)



1	2	1
0	0	0
-1	-2	-1

Horizontal
Sobel



**Horizontal Edge
(absolute value)**

Slide credit: David Lowe (UBC)

Horizontal Gradient

0	0	0
-1	0	1
0	0	0

or

-1	0	1
----	---	---

Vertical Gradient

0	1	0
0	0	0
0	-1	0

or

-1
0
1

Slide credit: David Lowe (UBC)

Il kernel può contenere valori negativi. E anche valori non interi.

Non è un caso limite si usa spesso.

E quindi l'immagine risultante? Non avevamo detto che usavamo solo valori non negativi?

- We can have negative values
- Generally pixel channels are unsigned
- How to handle them?
 - Clamping: put negative values as 0
 - Abs: consider absolute values only
 - Shift: add a value (may reduce interval)
 - Use them: adapt internal representation to handle them

Se uso dei “pesi” negativi potenzialmente potrei ottenere valori negativi anche nel risultato... Come li gestisco? Innanzitutto non è detto abbia senso doverli adattare. Il risultato può essere a sua volta una immagine ma anche no.

Diciamo che spesso però è comodo sia una immagine per visualizzazione (altrimenti non potevo mostrarvi gli “edge” del buon Albert...) per cui può aver senso rimuovere quei -

Qui vi si elencano alcune strategie. A parte la quarta opzione (che va letta come “me li tengo”) nelle altre spesso perdo informazione.

- **Some** kernel filters are **separable**
- $n \times n$ kernel in a single convolution
 - $O(M \times M \times n \times n)$
- $1 \times n$ and $n \times 1$ convolutions
 - $O(M \times M \times (n+n))$
- Mean filtering example:

$$\frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * \frac{1}{3} [1 \quad 1 \quad 1] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Le operazioni di convoluzione sono onerose. In quanto devo ripetere per ciascun pixel dell'immagine ($M \times M$) e per ciascuno di questi guardare il vicinato (che dipende dalle dimensioni del kernel $n \times n$)

Molti filtri sono però “separabili”, come nell'esempio in esame invece di applicare un singolo kernel 3×3 ne applico due, uno 1×3 e un secondo 3×1 . Il risultato è lo stesso ma riduco le operazioni da fare di $1/3$ (e sarebbe molto di più per kernel più “grossi”)

IMAGE FILTERING
MAIN LOCAL OPERATIONS



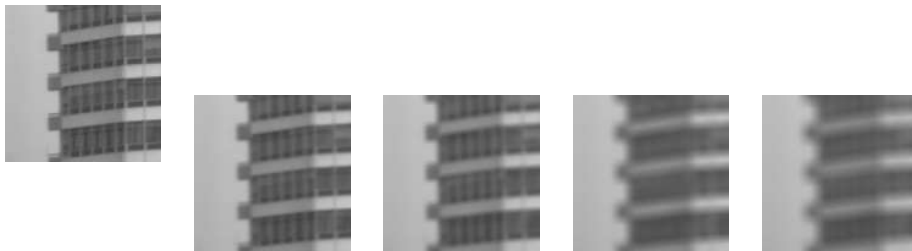
- Image can be affected by noise
 - Sensor
 - Transmission
 - Lossy compressions
 - Filtering can enhance noise (derivative operations)
- Different kind of noises
 - White noise
 - Salt&Pepper
 - ...
- Smoothing is often used to “reduce” it



- Averaging neighborhood can smooth images
- The bigger the kernel size, the augmented the smooth effect
 - Balancing act
 - Examples with 3x3, 5x5, 9x9, and 11x11 kernels

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



Lo abbiamo già visto, è in pratica una media su una finestra $n \times n$. Aumentando n aumenta anche l'effetto smoothing. Eliminiamo dei dettagli. È un filtro passa basso.

- Pros
 - Easy implementation
- Cons:
 - Weight of each pixel is the same
 - Not good for specific noise effects
 - Introduce high frequency artifacts

Una media è semplice da implementare. Però di fatto non tengo conto della distanza dei pixel di vicinato da quello su cui il kernel è “centrato”. Oltretutto il kernel è tipicamente quadrato. I pixel negli angoli pesano uguale ai pixel sui bordi...



Noisy Image



“Ideal” Filtering



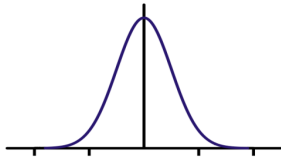
Mean Filtering result 👎

Non risolve tutte le situazioni, ad esempio il rumore “sale e pepe” lo vorrei direttamente eliminare e non tenerne conto ai fini media.



- Mean filtering kernel use the same weight for each pixel
 - But the distance is different
- Gaussian filtering gives more weight to the central pixel and:
 - The farther away the neighbors, the smaller their contribution
 - A gaussian is used

Una media è semplice da implementare. Però di fatto non tengo conto della distanza dei pixel di vicinato da quello su cui il kernel è “centrato”. Oltretutto il kernel è tipicamente quadrato. I pixel negli angoli pesano uguale ai pixel sui bordi...

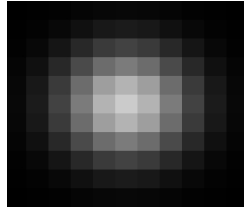


$$g_{\sigma}(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



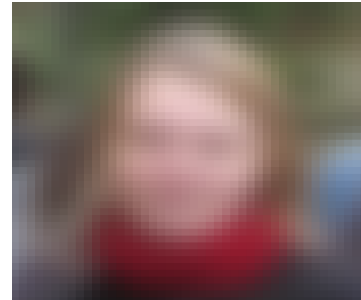
Input image f

*



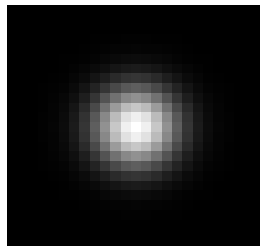
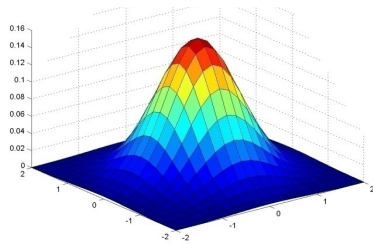
Filter g

=



Output image h

U e v sono le “coordinate” dei pesi del kernel con 0,0 nel centro. All’aumentare del loro valore diminuisce $g()$

Spatially-weighted average

0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$
(normalized values)

$$g_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

Slide credit: Christopher Rasmussen

u e v sono le “coordinate” dei pesi del kernel con 0,0 nel centro. All’aumentare del loro valore diminuisce g()
 Però una gaussiana si estende all’infinito, il kernel no.
 Quindi, oltre ad applicare la formula devo normalizzare.
 In quel kernel 5x5 i pesi non sono quelli direttamente calcolati grazie alla formula ma sono stati “aggiustati” di modo da ottenere 1 come loro somma (come lo sarebbe l’integrale della gaussiana da $-\infty$ a $+\infty$)



Nel caso del mean box si possono notare vignettature verticali e orizzontali mentre nel caso del gaussiano non ci sono

SMOOTHING: Gaussian vs Mean

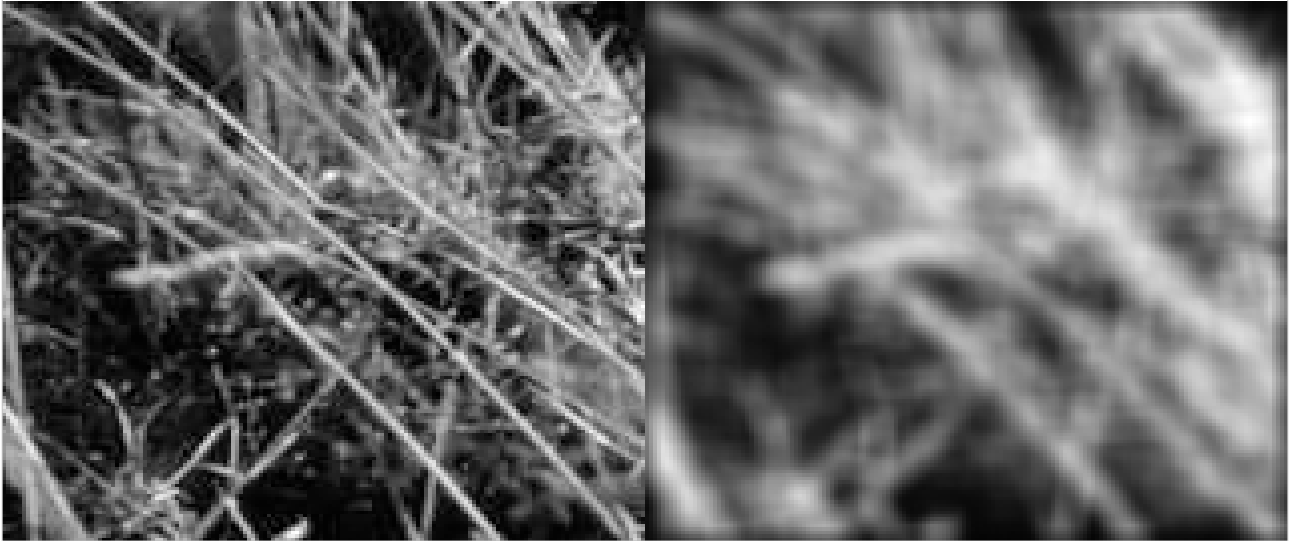
UNIVERSITÀ
DI PARMA

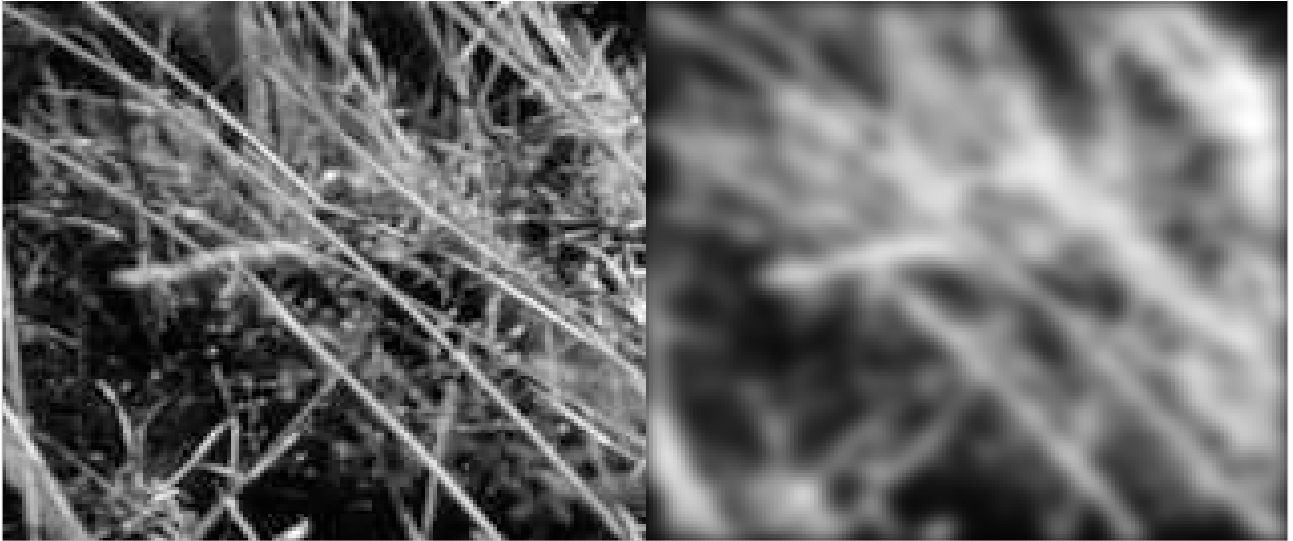


Slide credit: Robert Collins

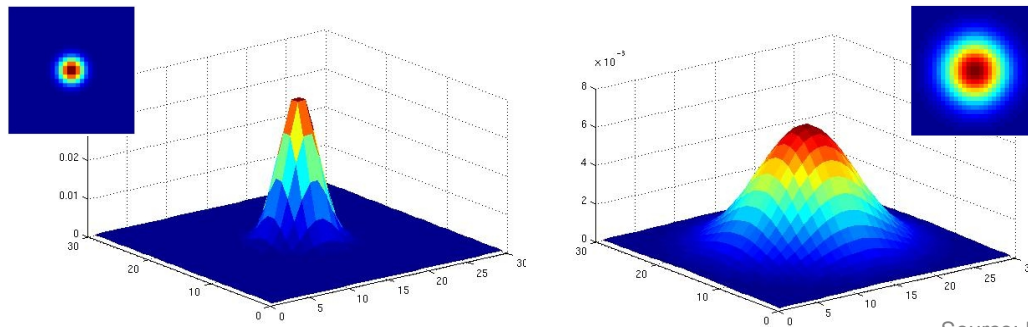
SMOOTHING: mean filtering artifacts

UNIVERSITÀ
DI PARMA





What parameters matter here?
Variance of Gaussian: determines
extent of smoothing

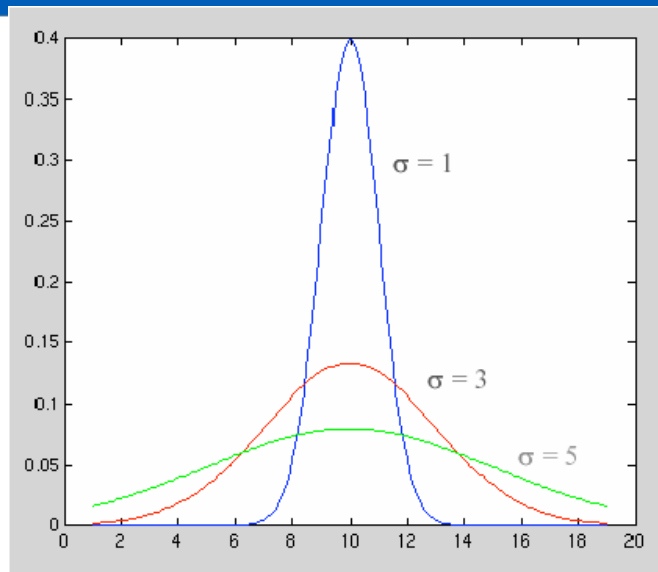


Source: K. Grauman

Il sigma mi determina l'andamento della gaussiana

What parameters matter here?

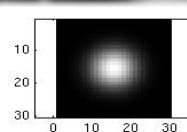
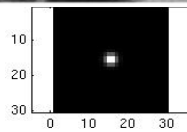
Variance of Gaussian:
determines extent of
smoothing



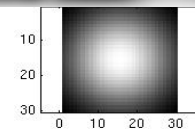
Il sigma mi determina l'andamento della gaussiana.
Aumentarlo significa aumentare il peso dei pixel
“lontani”, diminuirlo significa aumentare l'importanza
del pixel “centrale”



Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.



...



Source: K. Grauman

Look at edges... Do you see something?

$$\mathcal{G}_\sigma = \mathcal{G}_\sigma^x * \mathcal{G}_\sigma^y$$

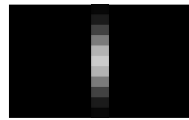
$$\mathcal{G}_\sigma^x(x, y) = \frac{1}{Z} e^{-\frac{x^2}{2\sigma^2}}$$

$$\mathcal{G}_\sigma^y(x, y) = \frac{1}{Z} e^{-\frac{y^2}{2\sigma^2}}$$

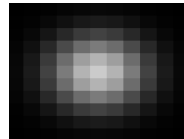
Gaussian filter is **separable**: compute in **horizontal** direction, followed by the **vertical** direction.



*

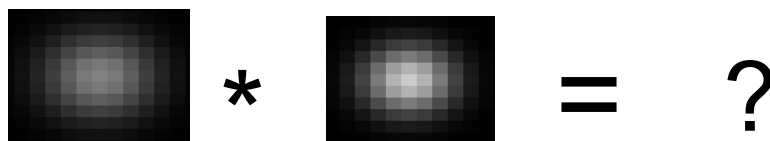


=



Faster!

Anche il filtro gaussiano è separabile



$$f * \mathcal{G}_\sigma * \mathcal{G}_{\sigma'} = f * \mathcal{G}_{\sigma''}$$

$$\sigma'' = \sqrt{\sigma^2 + \sigma'^2}$$

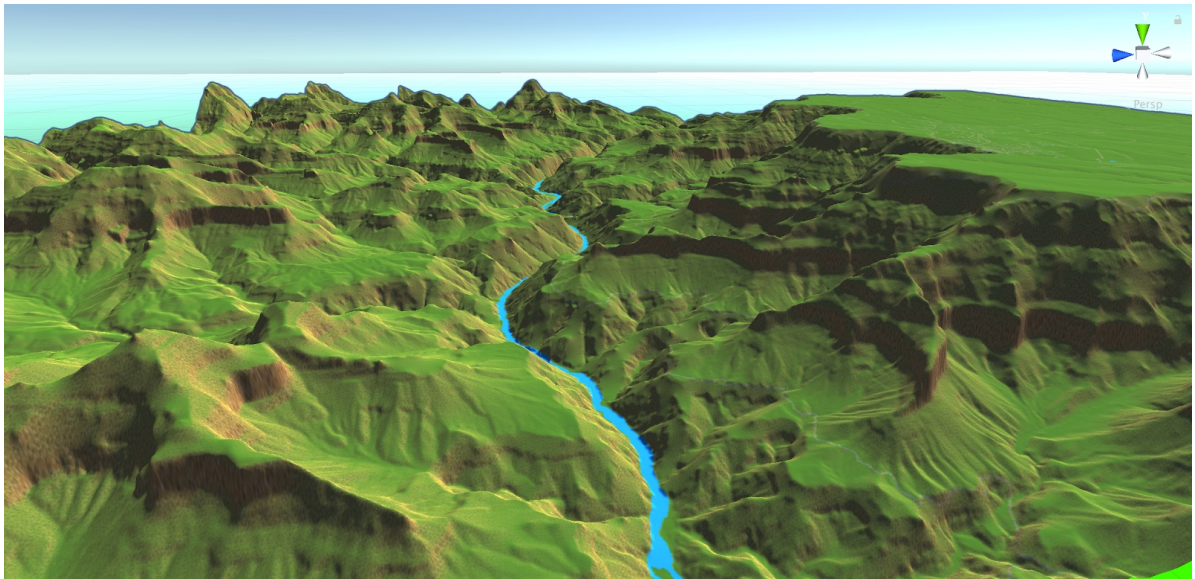
Cascade Gaussians: we can use smaller kernels

Con sigma alti avrebbe anche senso usare kernel più grandi (altrimenti si cade nel letterbox). Però è costoso computazionalmente.

Vediamo cosa succede se combino due convoluzioni consecutive con due sigma diversi (ma ovviamente la formula è la stessa con sigma uguali)

All'atto pratico è come usare una singola convoluzione con un sigma più alto

Questo mi permette di fare convoluzioni con kernel più piccoli e ottenere lo stesso risultato di un kernel più grande (al netto di qualche approssimazione dovuta alle dimensioni finite dei kernel)



Pensiamo all'immagine come funzione come già accennato. Abbiamo detto che al nero corrisponde 0, a pixel più chiari valori > 0

Ad eccezione di immagini che contengano rumore e basta è ragionevole che ci sia un po' di coerenza, ovvero avremo aree più o meno uniformi e comunque passaggi spesso non immediati da chiaro e scuro e viceversa.

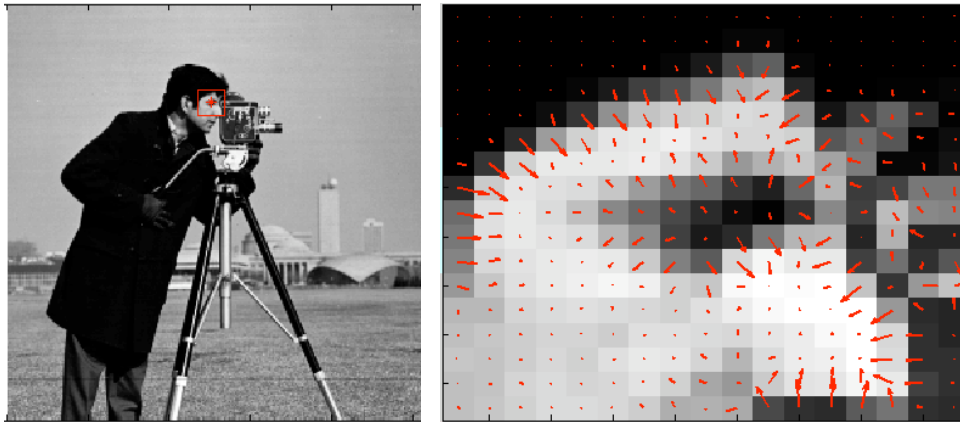
Posso in un certo senso vedere la mia immagine come una mappa di altitudine geografica. In cui esisteranno sicuramente avvallamenti o rilievi decisi (passaggio immediato chiaro/scuro) ma anche aree in cui ha senso parlare di salite e discese più o meno dolci in determinate situazioni.

In entrambe le situazioni ha senso parlare di gradiente ovvero della misura di come cambia l'immagine spostandoci su di essa.

- We can see images as 2D “surfaces” where pixel “intensity” encodes elevation
- Therefore we can use concepts like:
 - Uphill/Downhill
 - Peaks/Valleys
 - Discontinuities/Steepness
- Do you remember we can see images as functions?
 - $I(x,y)$
- For each (x,y) what function can say us something about local intensity variations?
 - Gradient

$$\nabla I(x,y) = \begin{bmatrix} \frac{\partial I(x,y)}{\partial x} \\ \frac{\partial I(x,y)}{\partial y} \end{bmatrix}$$

E il gradiente matematicamente è una derivata.
Nel caso di funzioni 2D di fatto un vettore le cui componenti orizzontale e verticale sono le due derivate in direzione orizzontale e verticale



Credits: Robert Collins

Questa immagine ingrandita mostra i vettori gradiente. Il vettore codifica sia la direzione in cui il gradiente varia maggiormente che l'entità della variazione in base al modulo.

- For each pixel we have a vector
 - Magnitude
 - Direction
- Images are discrete functions
 - How we can compute numerical derivatives?

$$\frac{\partial I(x, y)}{\partial x} = \lim_{h \rightarrow 0} \frac{I(x+h, y) - I(x, y)}{h}$$

Sappiamo tutti (beh lo sapevamo almeno a suo tempo quando le studiammo) come si calcolano le derivate nel continuo. L'immagine però è discreta (le coordinate di ciascun pixel sono intere). Come calcolo quindi la derivata?

Ricordiamoci come nasce la derivata ovvero è il limite mostrato nella formula.

- Taylor Series Expansion can be used
- Precision of the following formula is proportional to h^2

$$\frac{\partial I(x, y)}{\partial x} \approx \frac{I(x+h, y) - I(x-h, y)}{2h} \quad \frac{\partial I(x, y)}{\partial y} \approx \frac{I(x, y+h) - I(x, y-h)}{2h}$$

Quel limite si può approssimare come indicato qui sopra (in sostanza è una espansione di Taylor in cui ometto tutti i termini dopo il primo nella sommatoria)

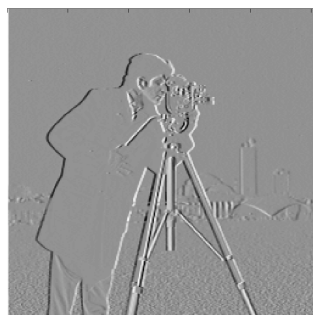
L'h più piccolo che possiamo usare è 1. Quindi i due gradienti li posso immaginare come la mera differenza dei pixel sopra e sotto oppure a destra e sinistra



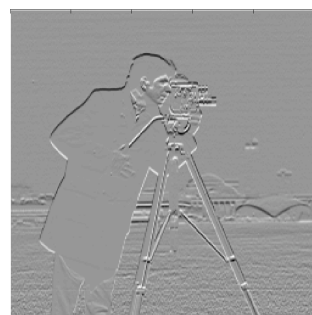
$$I(x, y)$$



$$I_x = \frac{I(x+1, y) - I(x-1, y)}{2}$$



$$I_y = \frac{I(x, y+1) - I(x, y-1)}{2}$$



Credits: Robert Collins

Qui un esempio

Per visualizzarlo si è dovuto però gestire i numeri negativi.

Il trucco usato è quello di scalare i valori incastrandoli nel range non negativo 0-255.

In pratica lo 0 originale (che ottengo nelle aree uniformi ovvero dove la differenza dei pixel vicini è nulla) diventa il grigino. Valori negativi diventano pixel scuri e valori positivi a pixel chiari.

Osservate la transizioni orizzontali del cappotto del fotografo e lo capite subito.



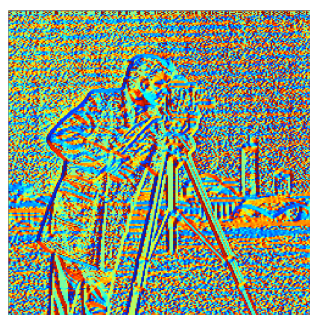
$$I(x, y)$$



$$\sqrt{I_x^2 + I_y^2}$$



$$\text{atan2}(I_y, I_x)$$



Credits: Robert Collins

Avevamo però detto che il gradiente è un vettore che ha come componenti il risultato delle due derivate.

Come lo raffiguro? Come freccina sarebbe simpatico ma è un po' complesso.

In queste due immagini vedete la sua ampiezza (magnitude) e gli angoli.

L'immagine degli angoli è molto rumorosa, ma dipende dal fatto che anche nelle aree uniformi dove di fatto ho un gradiente irrisorio si calcola comunque un angolo.

In effetti, per maggiore comprensibilità, andrebbero mostrati gli angoli solo dove c'è un'ampiezza significativa (negli esempi OpenCV lo faremo).

- I_x and I_y can be easily computed using simple operators

$$I_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} * I \quad I_x = \begin{bmatrix} +1 & 0 & -1 \end{bmatrix} * I$$

- Problem: noise

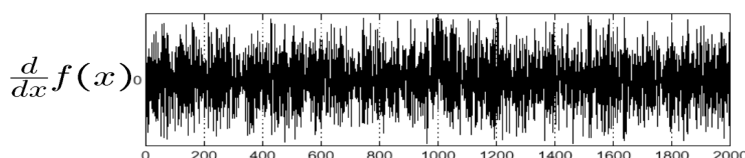
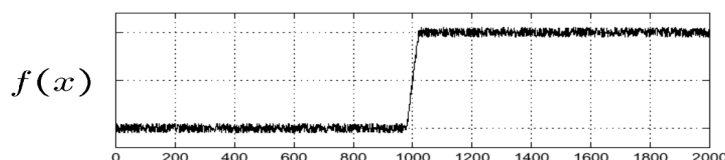
Per $h=1$ le formule viste due slide prima si riducono alla differenza dei pixel destra/sinistra e sopra/sotto.

Di fatto le posso vedere come convoluzioni usando kernel 1×3 e 3×1

Piccolo problema, la presenza di rumore viene esaltata quando derivo...



Consider a single image row



Finding the “edge” is not so easy...

Source: A. Bobick

Per semplicità consideriamo una funzione monodimensionale. Immaginiamo un andamento in cui ho un bel salto di valori, ma comunque ho del rumore.

Il fatto è che il salto è molto ben evidente, ampio ma allo stesso tempo “dolce”. Il rumore invece influisce poco su valore ma è però molto più repentino.

Purtroppo la derivata tende a pesare il secondo come il primo.

Nota: sì è una situazione limite, ma era per far capire...

- Usually we want to find *strong* gradient variations (edges)
- How to remove noise? → smoothing

$$I_y = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} * ([1 \ 1 \ 1] * I) \quad I_x = [+1 \ 0 \ -1] * \left(\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * I \right)$$

Come rimuovo il rumore? Lo abbiamo già visto, un filtro di smoothing. Orizzontale per gradienti verticali e viceversa. Di questo modo invece di pesare la sola differenza, ad esempio, sopra/sotto guardo la differenza di una media locale dei pixel adiacenti a quelli sopra e sotto.

- Usually we want to find *strong* gradient variations (edges)
- How to remove noise? → smoothing

$$I_y = \begin{bmatrix} +1 & 0 & -1 \\ +1 & 0 & -1 \\ +1 & 0 & -1 \end{bmatrix} * I \quad I_x = \begin{bmatrix} +1 & +1 & +1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} * I$$

- Prewitt

Il risultato della moltiplicazione di quei due kernel sono i kernel qui raffigurati.

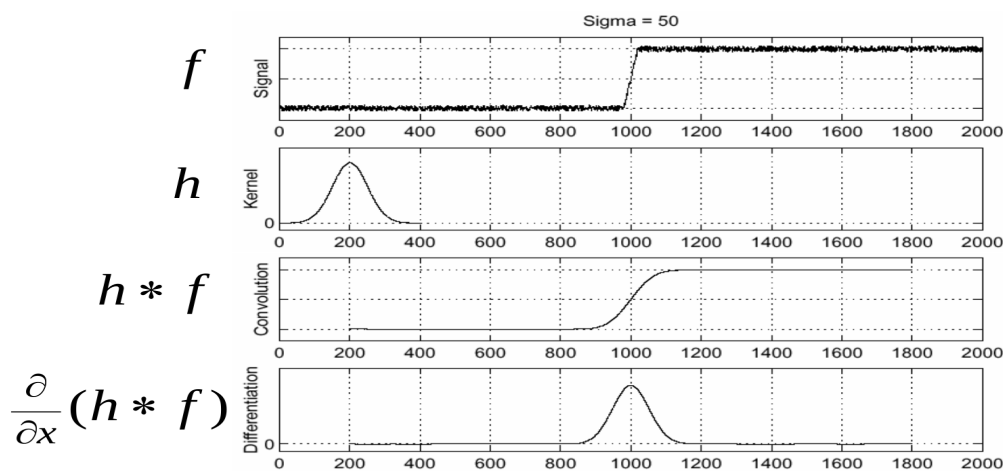
Questo è il cosiddetto filtro di prewitt (Prewitt, J.M.S. (1970). "Object Enhancement and Extraction". *Picture processing and Psychopictorics*. Academic Press.)

- As seen for smoothing for prewitt operators we have the same weight
- **Sobel** operators slightly improves this

$$I_y = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * I \quad I_x = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * I$$

- For smoothing we also saw a gaussian approach

Dare lo stesso peso ai tre pixel sopra/sotto o destra/sinistra non tiene conto della distanza. Un buon compromesso è pesare comunque di più i pixel “centrali” da cui il filtro di Sobel (1973)



Where is the maximum gradient?

Source: A. Bobick

E se per lo smoothing applicassimo un filtro gaussiano?

Ancora una volta, esempio in 2D

F: immagine in ingresso

H: filtro Gaussiano

Applicato "sbiotto" prima faccio convoluzione e poi derivo



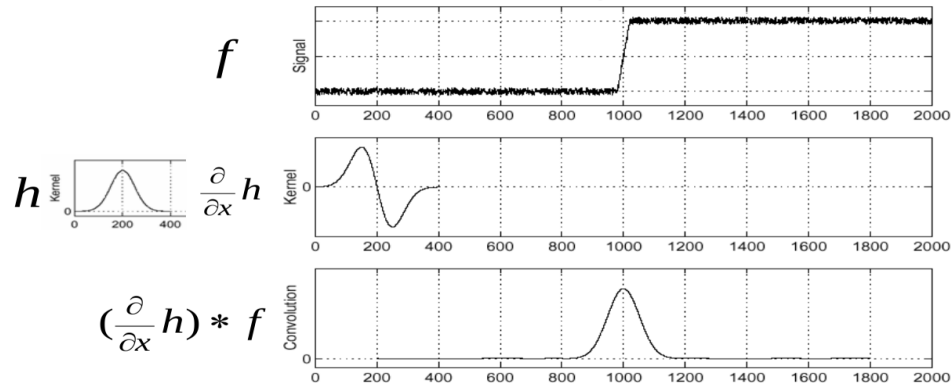
- Introducing Prewitt/Sobel we distributed the convolution
 - Namely we applied the derivative operator to smoothing one
- Derivative theorem of convolution

$$\frac{\partial}{\partial x}(f * g) = \left(\frac{\partial}{\partial x} f \right) * g = f * \left(\frac{\partial}{\partial x} g \right)$$

MA all'atto pratico posso derivare il solo kernel (piccolo e quindi ci spendo poco) e poi fare la convoluzione con deciso vantaggio computazionale.

Posso farlo? Sì, c'è un teorema della convoluzione che me lo permette e, se ci pensate, è la stessa cosa che abbiamo fatto con Sobel/Prewitt

- This saves us one operation: $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x}h) * f$



And if we want to find local maxima?

Source: A. Bobick

In pratica derivo h e uso il risultato per derivare

- 1st derivative filters
- Where is the edge?
 - Magnitude peaks

$$\sqrt{\frac{\partial}{\partial x} I(x, y)}$$

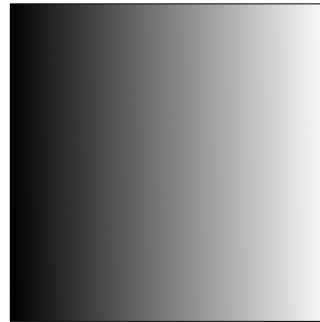


Passiamo ad altro. Come trovo dove ho i massimi del gradiente ovvero dove realisticamente ho un bordo forte (ritorneremo dopo sulla definizione di edge)?

Se ci sono ovviamente.

- 1st derivative filters
- Where is the edge?
 - Magnitude peaks

$$\sqrt{I_x^2 + I_y^2} > Th$$



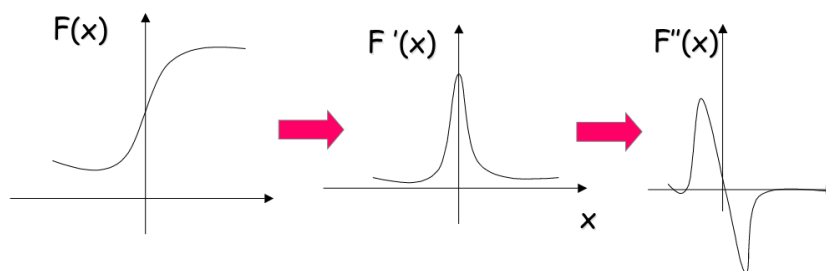
Una possibilità è l'uso dell'ampiezza del gradiente.

Dove è al di sopra di un certo valore assumo ci sia una forte variazione della luminosità.

Questo valore viene chiamato “soglia” (threshold in inglese)



- Using 1st derivative filters maxima detect edges
 - Not simple to exactly find them
- What about 2nd derivative filters?



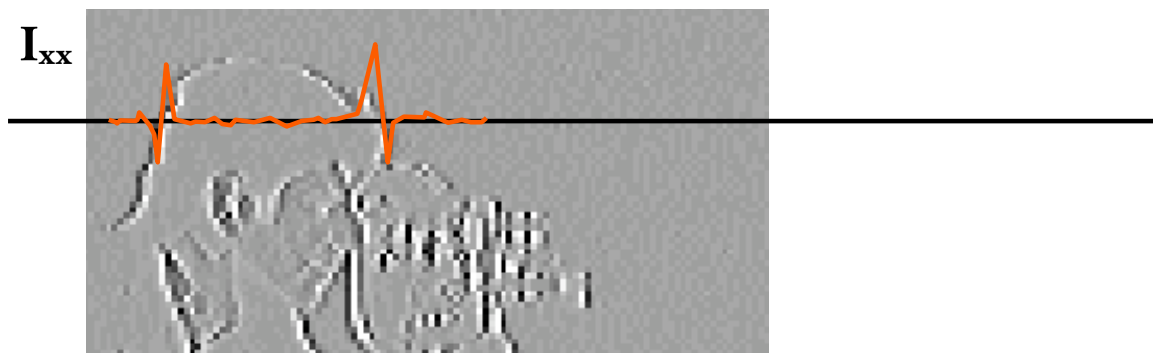
Source: R. Collins

Comunque trovare questi massimi non è semplicissimo o meglio diventa complesso se ragiono in aree locali dell'immagine.

Una seconda possibilità è l'uso della derivata seconda. In questo caso il massimo della derivata prima diventa uno 0 nella derivata seconda. Relativamente facile da trovare visto che posso guardare le transizioni da + o - o viceversa



- Pro: easy localization of edges
- Cons: 2nd derivative is 0 also when 1st one is 0



Source: R. Collins

È più facile che trovare i massimi ma devo evitare come la peste di usarlo in aree uniformi ovvero dove la magnitudine della derivata prima è pressoché nullo ma che nella derivata seconda si trasforma in abbondanti 0

Soluzione: considero gli zeri solo dove la derivata prima ha sufficiente magnitudine

- Laplacian operator ∇^2 can be used to combine I_{xx} and I_{yy}

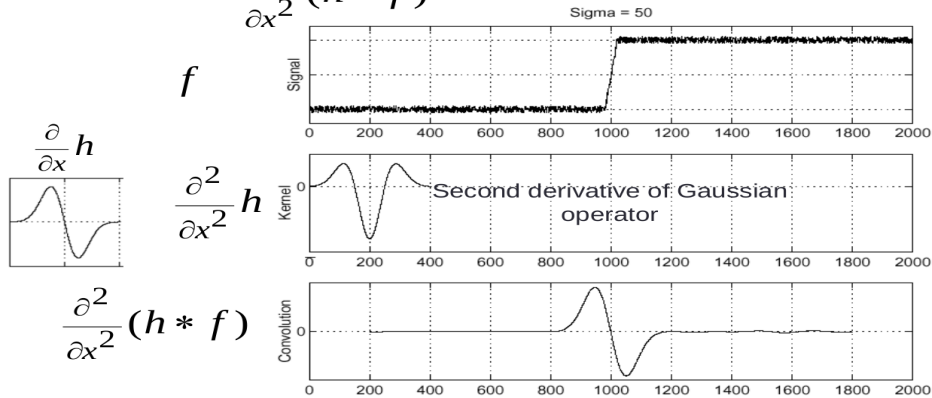
$$\nabla^2 I(x, y) = I_{xx}(x, y) + I_{yy}(x, y) \approx \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} * I$$



Source: R. Collins

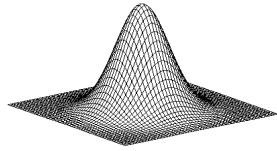
Ecco un esempio di operatore che combina le derivate seconde → Operatore Laplaciano
Si noti il passaggio da nero a bianco dove ho gli edge più significativi

- Consider $\frac{\partial^2}{\partial x^2}(h * f)$



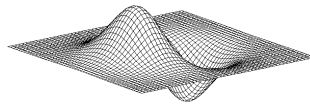
The maxima is where we have a zero crossing

Source: A. Bobick



Gaussian

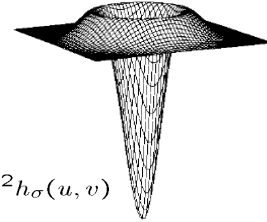
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian
or LoG filter

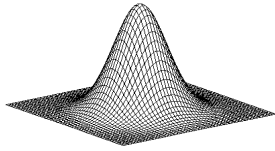


$$\nabla^2 h_{\sigma}(u, v)$$

∇^2 is the Laplacian operator (sum of 2nd derivatives):

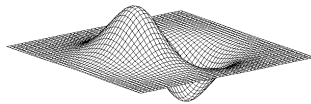
$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Una rappresentazione come funzione $\mathbb{R}^2 \rightarrow \mathbb{R}$



Gaussian

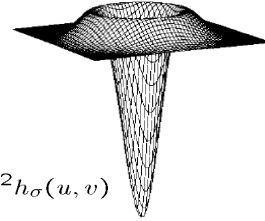
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

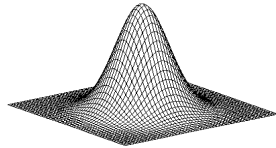
$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian
or LoG filter



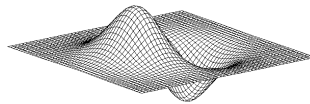
$$\nabla^2 h_{\sigma}(u, v)$$

$$\begin{aligned} &\text{Laplacian(Gaussian * f)} \\ &= \\ &\text{LoG * f} \end{aligned}$$



Gaussian

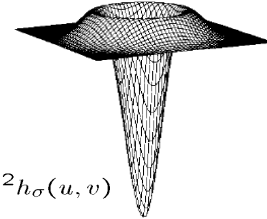
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



x derivative of
Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian
or LoG filter



$$\nabla^2 h_{\sigma}(u, v)$$

$$\nabla^2 G(x, y; \sigma) = \left(\frac{x^2 + y^2}{\sigma^4} - \frac{2}{\sigma^2} \right) G(x, y; \sigma)$$

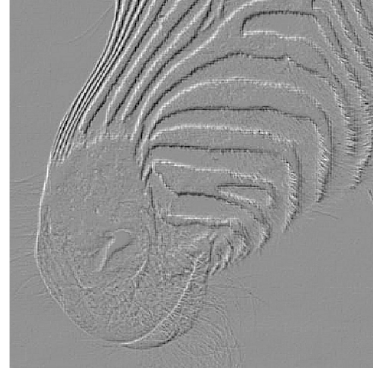
Often approximated
by

0	1	0
1	-4	1
0	1	0

C'è anche la versione con tutti 1 e -8 oppure la negativa



- Band pass filter
 - Suppress low frequencies (2^{nd} derivative) but also high ones (gaussian)



Source: R. Collins

Questo operatore che di fatto combina gaussiana e derivativa di fatto è un passa banda. Sopprime le frequenze alte (componente gaussiana) ma anche le basse (componente derivativa)

- Used as “edge” detector, extract “raw” zero crossings:
 - Different sigmas (2 & 4)
 - Like a “equal elevation” on a map



Source: R. Collins

Analizza altri usi come l'individuazione di blob e tecniche di compressione lossy in cui un LoG viene applicato ad una piramide e guadagno rispetto a comprimere l'immagine intera

- We now should have understood basic convolution principles
- We can add further complexity
 - Border issues
 - Padding
 - Stride

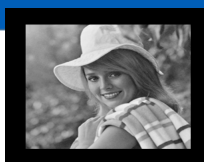
Qualche dettaglio della convoluzione l'abbiamo, per semplicità tralasciato, ma è il caso comunque di discuterne

- How we deal with border/corner pixels in a convolution?
 - Actually the same issue for other operations that involve a mask/kernel
- Basically two approaches
 - Result of the processing is smaller than original image
 - Same size with border/corner pixels with *unknown* value

Primo problema: i bordi.

Abbiamo detto che l'operazione di convoluzione consiste, all'atto pratico, nel sovrapporre il kernel man mano a tutti i pixel dell'immagine ed eseguire una moltiplicazione e somma dei valori sovrapposti.

Cosa faccio con i pixel di bordo in cui non riesco a sovrapporre?



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

0	10	20	30	30	30	20	10
0	20	40	60	60	60	40	20
0	30	60	90	90	90	60	30
0	30	50	80	80	90	60	30
0	30	50	80	80	90	60	30
0	20	30	50	50	60	40	20
10	20	30	30	30	30	20	10
10	10	10	0	0	0	0	0

?	?	?	?	?	?	?	?	?	?
?	0	10	20	30	30	30	20	10	?
?	0	20	40	60	60	60	40	20	?
?	0	30	60	90	90	90	60	30	?
?	0	30	50	80	80	90	60	30	?
?	0	30	50	80	80	90	60	30	?
?	0	20	30	50	50	60	40	20	?
?	10	20	30	30	30	30	20	10	?
?	10	10	10	0	0	0	0	0	?
?	?	?	?	?	?	?	?	?	?

Pensiamo al primo esempio che abbiamo fatto. Avevamo usato un filtro di media 3x3. Ma avevamo ignorato i bordi, visto che non potevo completamente sovrapporre il kernel centrato su questi perché “sforerebbe” dall’immagine.

Quindi di fatto riesco a calcolare un risultato valido solo per una porzione dell’immagine ottenendo o una immagine più piccola oppure una immagine di dimensioni uguali a quella in ingresso ma con valori non definiti nei pixel di bordo.

Poi in questo caso il kernel era piccolo, ma il problema diventa più significativo per kernel più grandi e soprattutto se si applicano diversi filtri.

- **Padding** is a technique to “enlarge” images adding a given amount of pixels to each image border
- The most common approach is the **zero padding**
 - Added pixels are set to 0

Avere immagini risultanti di dimensioni differenti da quelle in ingresso può essere scomodo o fornire immagini non aderenti a specifici standard di dimensione (esempio se elaboro una immagine 4k che ha dimensioni ben precise, molto probabilmente vorrei anche un risultato con le stesse dimensioni).

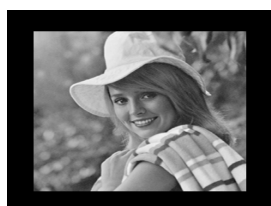
Non esiste una soluzione esatta. Per i pixel di bordo non avrò mai un risultato corretto.

Una possibilità che però mi salva dal problema dimensioni è quello di effettuare padding.

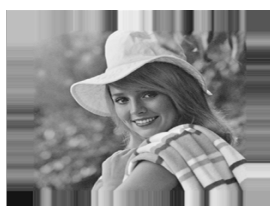
Elaborando tolgo una porzione dell'immagine originale? Prima di elaborare aggiungo all'immagine originale opportuno “bordo” di modo da allargarla e far sì che il risultato si delle stesse dimensioni di quella in ingresso → padding



- Padding is often use before a convolution to obtain same size result
- The “extra” pixels can be set using different policies
 - Again: zero padding is the most used



black



fixed



periodic



reflected

Differenti tecniche:

0 padding: aggiungo 0, è la più usata per mere questioni di semplicità. Rende però evidenti effetti di bordo

Fixed: replicò le righe colonne di bordo, anche questa spesso usata in quanto riduce gli effetti di bordo.

Periodic e reflected: si capisce dalle immagini, poco usate e, a mio avviso, poco sensate

- For a 4x4 image

20	128	60	100
50	66	72	20
73	98	65	89
20	170	190	200

- Zero padding 1 $\rightarrow$ output size is **6x6**

0	0	0	0	0	0
0	20	128	60	100	0
0	50	66	72	20	0
0	73	98	65	89	0
0	20	170	190	200	0
0	0	0	0	0	0

- When we use a 3x3 kernel we obtain a 4x4 resulting image

- Zero padding 2 → output size is **8x8**

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	20	128	60	100	0	0
0	0	50	66	72	20	0	0
0	0	73	98	65	89	0	0
0	0	20	170	190	200	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Ovviamente è un esempio molto limite non avrò mai immagini così piccole

- The stride is a parameter used to set kernel movements on the image
- Since now we assumed that kernel is horizontally and vertically moved on each row/column $\rightarrow$ stride is 1
- We can move a kernel skipping some pixel $\rightarrow$ stride > 1
- A stride > 1 leads to subsampling images!
- Moreover we can have different strides for rows/columns

Fino ad ora abbiamo assunto di considerare per il filtraggio tutti i punti dell'immagine in ingresso spostando quindi il kernel su tutti.

Quindi dapprima parto con il punto in alto a sinistra, poi mi sposto su quello di fianco a dx e così via. A fine riga ripartro dall'inizio della riga sotto ecc. ecc. Spostandomi quindi di 1 pixel alla volta.

Nulla vieta di saltarne alcuni. In pratica di quanto spostato il kernel si chiama "stride" (che in italiano significa "passo"). Per esempio con stride di 2 salto un pixel sia in orizzontale che in verticale.

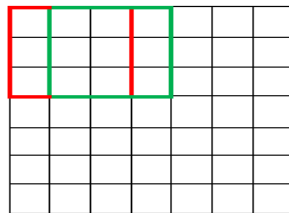
Penso sia intuibile che equivale ad avere una immagine risultante sottocampionata.

Attenzione che è differente dal sottocampionare e poi filtrare con stride = 1. Perché anche se salto dei pixel ne tengo comunque conto nei conti ;)

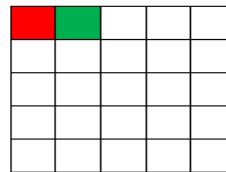


- Stride = 1 → “normal convolution”. Kernel is moved on every pixel (vertically & horizontally)
- I.e for 7x7 image, a 3x3 kernel and a stride value as 1 we will obtain a **5x5 image**

7 x 7 Input Volume



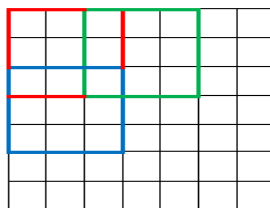
5 x 5 Output Volume





- Stride = 2 $\rightarrow$ kernel is moved skipping 1 column for each horizontal move and 1 row for each vertical move
- I.e for 7x7 image, a 3x3 kernel and a stride value as 1 we will obtain a **3x3 image**

7 x 7 Input Volume



3 x 3 Output Volume



- Given a image featuring n columns
- Using a kernel with f columns, with padding p and stride s , the number of columns of the output image is:

$$n_{new} = \lfloor \frac{n + 2 * p - f}{s} + 1 \rfloor$$

- The result must be the largest integer value equal or smaller than n_{new}
- The same formula can be used for computing the new number of rows

Stride a padding complicano un pochetto capire quali dimensioni possa avere l'immagine risultante. Qui una formula (i due operatori che sembrano parentesi quadre indicano che va presa la parte sola intera del risultato)

IMAGE FILTERING
LOCAL & NON LINEAR
OPERATIONS

- Since now we illustrate the so-called **linear filters**
 - The result is a linear sum of neighborhood values
 - Linear filters can easily be combined:
 - The result of the processing of the sum of 2 signals can be obtained as the sum of the processing of the 2 signals

$$S(\alpha * f[x, y] + \beta * g[x, y]) = \alpha * S(f[x, y]) + \beta * S(g[x, y]) \text{ for all } \alpha, \beta \in \mathbb{R}$$

- Linear filters can be easily combined and can also be analyzed in the frequency domain (Fourier transform) but they are not always the most efficient solution

Quelli visto fino ad ora erano i filtri lineari. E in effetti erano la combinazione di operazioni di somma e moltiplicazione.

Facili da usare e combinare, posso addirittura spostarmi nel dominio delle frequenze e usarli lì (trasformata di Fourier che noi non vedremo e che è invertibile)

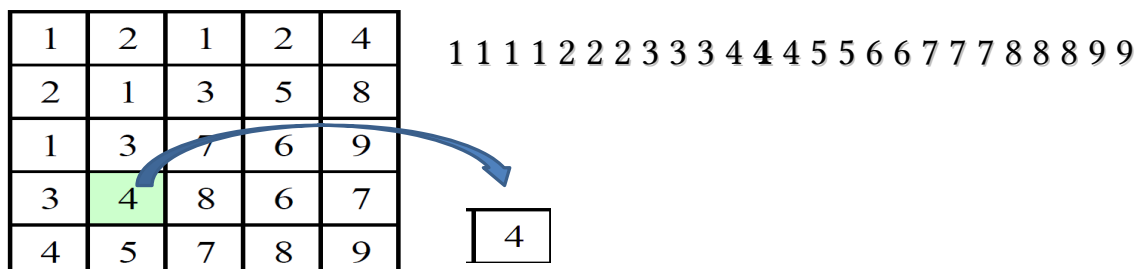
Ne esistono altri che si possono usare in ben specifiche condizioni in cui i filtri lineari non funzionano



- Median pooling
- Max/Min pooling
- *Average pooling*
- Bilateral filter



- For each neighborhood we select the median value



- Is a smoothing operation
- Can cope with salt & pepper noise
- Sorting needed → computationally expensive

Source: Szeliski

Filtro non lineare per eccellenza, ordino i valori del vicinato e il risultato è il valore a metà dell'elenco (la mediana). Nel caso in esame è il 4 in grassetto.

Di fatto è un filtro di smoothing. Cosa lo differenzia dal filtro di media già visto? Immaginate una immagine afflitta da rumore che non sia una semplice perturbazione dei valori dei pixel ma in cui diversi pixel sono saturi o neri (il cosiddetto rumore pepe e sale). Dei veri e propri "outlier" insomma. Applicare una media significherebbe far entrare nel risultato questi valori che vorrei del tutto eliminare.



Shot noise



5x5 average



5x5 median

Source: Collins

Con una mediana realisticamente li elimino in buona parte e l'immagine sopra lo evidenzia bene.

Piccolo problema. È molto pesante da calcolare, debbo infatti eseguire un ordinamento di tutto un vicinato per ciascun pixel dell'input e non esistono trucchi di ottimizzazione significativi.



- Select the largest/smallest value in the neighborhood

20	128	60	100
50	66	72	20
73	98	65	89
20	170	190	200

128	128	100
98	98	89
170	190	200

- Can be used to selectively remove salt & pepper noise

Molto usate in ambito DL

Se elimino massimi e minimi forse elimino outliers

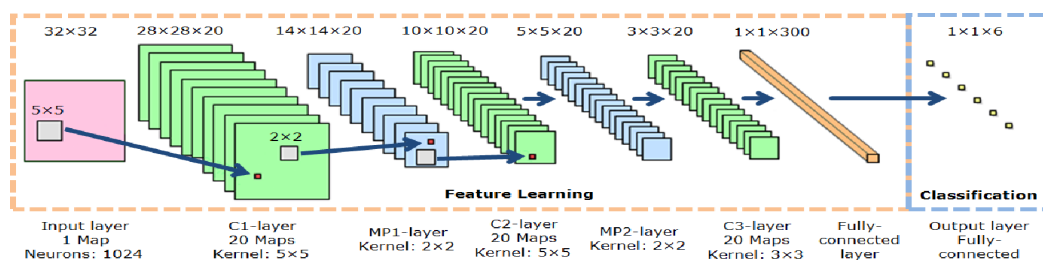
Un po' meno computazionalmente pesanti di un filtro mediano



- We already saw this guy!
- Box filter (linear)

In realtà questo è lineare

- Convoluzioni con stride e max pooling sono gli elementi base di una **Convolutional Neural Network (CNN)**:



La differenza è che le convoluzioni non sono staticamente decise dal programmatore ma in pratica è l'addestramento che le stabilisce ovvero dinamicamente

- Often convolutions imply to repeatedly compute the sum of a region.

$$S(i, j) = \sum_{k=0} \sum_{l=0} F(k, l)$$

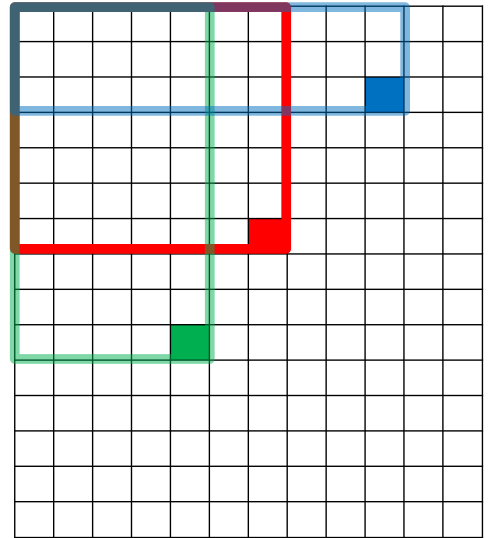
- This is an expensive operation $\rightarrow O(n^2)$
- The **integral image** can lead to $O(n)$
- Also known as **summed area table**

243	239	240	225	206	185	188	218	211	206	216	225
242	239	218	110	67	31	34	152	213	206	208	221
243	242	123	58	94	82	132	77	108	208	208	215
235	217	115	212	243	236	247	139	91	209	208	211
233	208	131	222	219	226	196	114	74	208	213	214
232	217	131	116	77	150	69	56	52	201	228	223
232	232	182	186	184	179	159	123	93	232	235	235
232	236	201	154	216	133	129	81	175	252	241	240
235	238	230	128	172	138	65	63	234	249	241	245
237	236	247	143	59	78	10	94	255	248	247	251
234	237	245	193	55	33	115	144	213	255	253	251
248	245	161	128	149	109	138	65	47	156	239	255
190	107	39	102	94	73	114	58	17	7	51	137
23	32	33	148	168	203	179	43	27	17	12	8
17	26	12	160	255	255	109	22	26	19	35	24

Per esempio nel caso del filtro di media devo fare la somma dei valori di un'area dell'immagine, per ogni sovrapposizione del kernel. Abbiamo visto che, nelle operazioni lineari, si raggiunge una complessità che dipende dal quadrato dell'immagine moltiplicato però per le dimensioni del kernel

Introduciamo l'immagine integrale che vedremo ci permette di aumentare, non poco, l'efficienza del tutto

- Each pixel (x,y) contains the sum of other pixels contained in the rectangle $(0,0)-(x,y)$



In pratica è una “immagine” delle stesse dimensioni dell’immagine di partenza ma in cui ogni pixel contiene la somma dei valori di tutti i pixel con coordinate di riga/colonna $\leq$ a quelle del pixel in esame

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4				

Partiamo a fare un esempio, il primo pixel rimane uguale...

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7			

Spostandomi lungo la riga sommo il primo pixel e il secondo

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14		

E così via



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	

Mi sembra chiaro che per la prima riga mi basta sommare il pixel in esame con il valore ottenuto per il pixel precedente, ovvero $9+14$ in questo caso

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26

E fino a fine riga funziona...



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6				

Ma quando parto con la riga successiva le cose si complicano. Devo quindi ogni volta ri-sommare tutti i pixel o c'è un sistema più semplice?

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14			

Sì, c'è: nell'immagine integrale sommo cella sopra + cella a sinistra - cella in alto a sinistra + pixel in immagine originale



- A fast way to compute the summed area table is:
 - Initialize integral image I using same values of original image
 - Start from (0,0) and move row by row
 - Update each “cell” as
 - $I(x, y) = I(x, y) + I(x-1, y) + I(x, y-1) - I(x-1, y-1)$

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

Questo trucco riduce non poco i conti che devo fare

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	50
11	25	43	68	74
12	27	49	77	85
12	29	54	86	95

In pratica per calcolare l'immagine integrale mi basta ciclare su tutti i pixel



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7	14	23	26
6	14	28	46	50
11	25	43	68	74
12	27	49	77	85
12	29	54	86	95

Let's try to compute the sum of that area $7+9+1+4+7+2+4+3+2 = 39$

Comunque il calcolo è sempre $O(M^2)$, dove sta l'efficienza?

Quanto ci metto a calcolare la somma dei valori dei pixel di un'area $n*n$? Senza l'immagine integrale sicuramente $n*n$, ma ora?

4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85 _A
12	29	54	86	95

Consider the same area on the Integral Image and cells A, B, C, D



4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85
12	29	54	86	95 _A

The result is $A+B-C-D \rightarrow 85 + 7 - 26 - 27 = 39$

Il pixel in A mi contiene la somma di tutti i valori dell'immagine originale nel quadrato rosso (A). Se vi sottraggo i valori dei pixel nell'area gialla (A-D) rimuovo il contributo di questi pixel. Idem se sottraggo la somma dei pixel contenuti nell'area blu (A-D-C).

Però con queste due sottrazioni rimuovo due volte il contributo dei pixel nell'area verde che devo quindi riaggiungere (A-D-C+B)

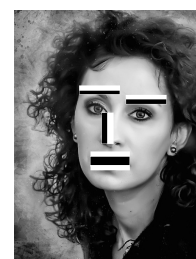
4	3	7	9	3
2	5	7	9	1
5	6	4	7	2
1	1	4	3	2
0	2	3	4	1

4	7 _B	14	23	26 _C
6	14	28	46	50
11	25	43	68	74
12	27 _D	49	77	85 _A
12	29	54	86	95

The result is $A+B-C-D \rightarrow 85 + 7 - 26 - 27 = 39$

Se ho l'immagine integrale questa operazione è di fatto indipendente dalle dimensioni dell'area per cui devo calcolare la somma $\rightarrow O(1)$

- Examples of usage
 - Filter box: given a $n \times n$ kernel on a $M \times N$ image
 - No integral image, separable kernels $\rightarrow O(M \times N \times 2n)$
 - Computing integral image $\rightarrow O(M \times N \times 2)$
 - Haar like features
 - Binary patterns used for face detection



Prima dell'avvento del DL uno dei sistemi più usati per il riconoscimento dei volti era quello di Viola Jones. Questo si basava nel sovrapporre alcuni pattern (Haar-like features) sull'immagine in tutte le posizioni possibili e in tutte le dimensioni possibili e calcolare la somma dei valori dei pixel per le aree nere e per le aree bianche. Senza l'immagine integrale era un'operazione estremamente onerosa, ma grazie a questo "trucco" era banalmente implementabile anche su HW modesti

